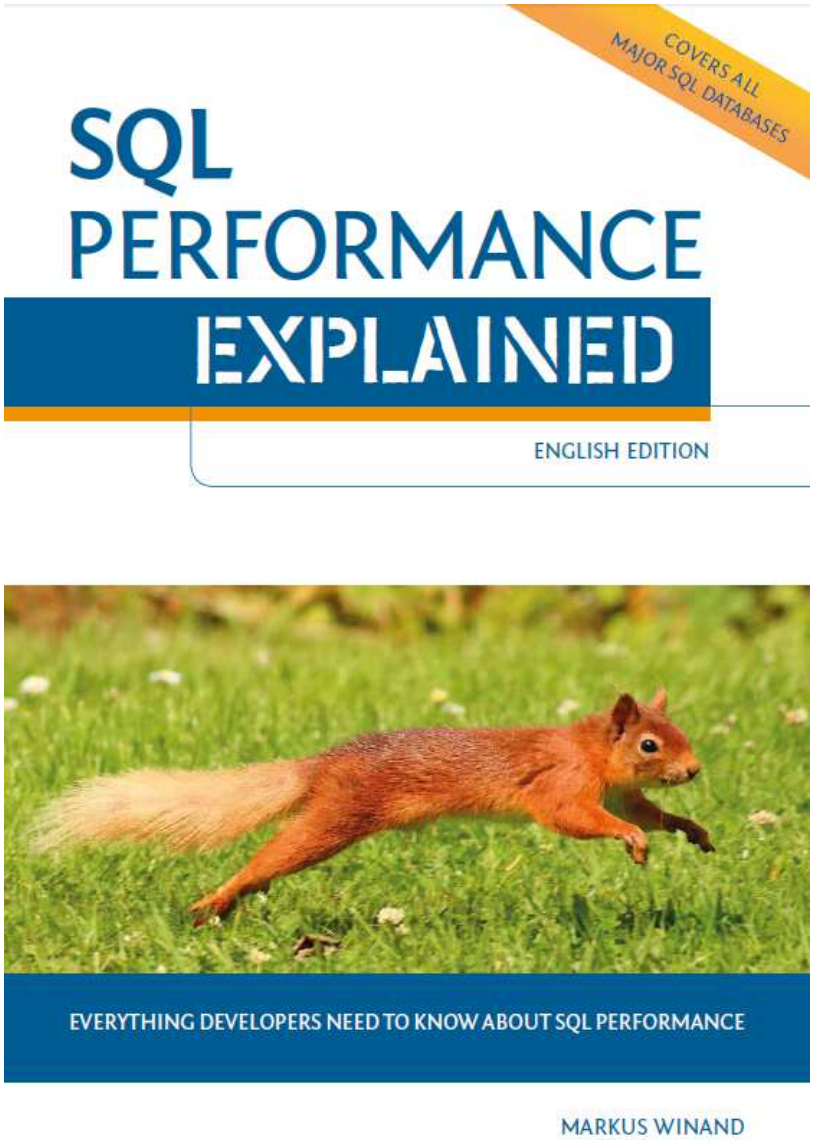
Practică SGBD

http://use-the-index-luke.com/



Performanța - Volumul de date

Don't ask a DBA to help you move furniture.
They've been known to drop tables…

Volumul de date

- O interogare devine mai lentă cu cât sunt mai multe date în baza de date

- Cât de mare este impactul asupra performanței dacă volumul datelor se dublează ?
- Cum putem îmbunătăți ?

Volumul de date

- Interogarea analizată:

```
SELECT count(*) FROM scale_data
  WHERE section = ? AND id2 = ?
```

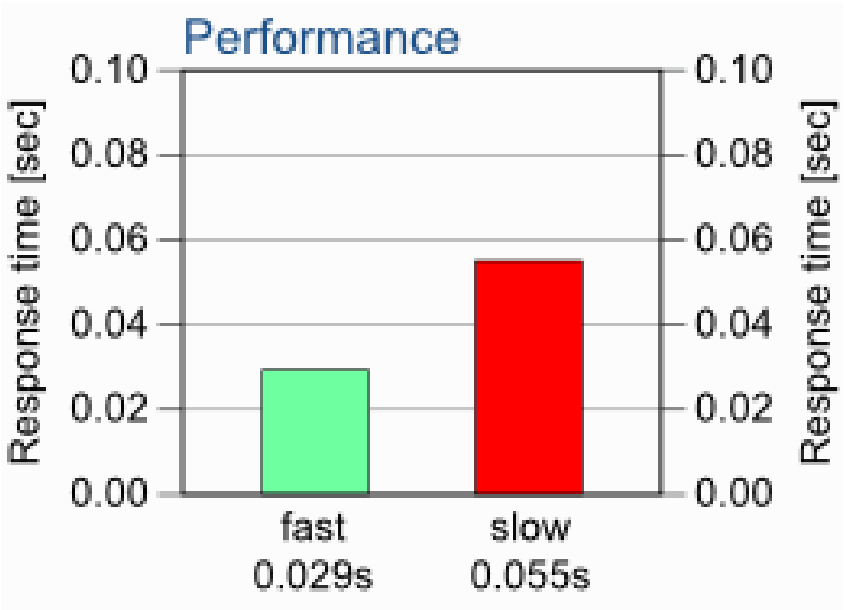
- Section are rolul de a controla volumul de date. Cu cât este mai mare section, cu atât este mai mare volumul de date returnat.
- Considerăm doi indecși: index1 și index2

Volumul de date

- Interogarea analizată:

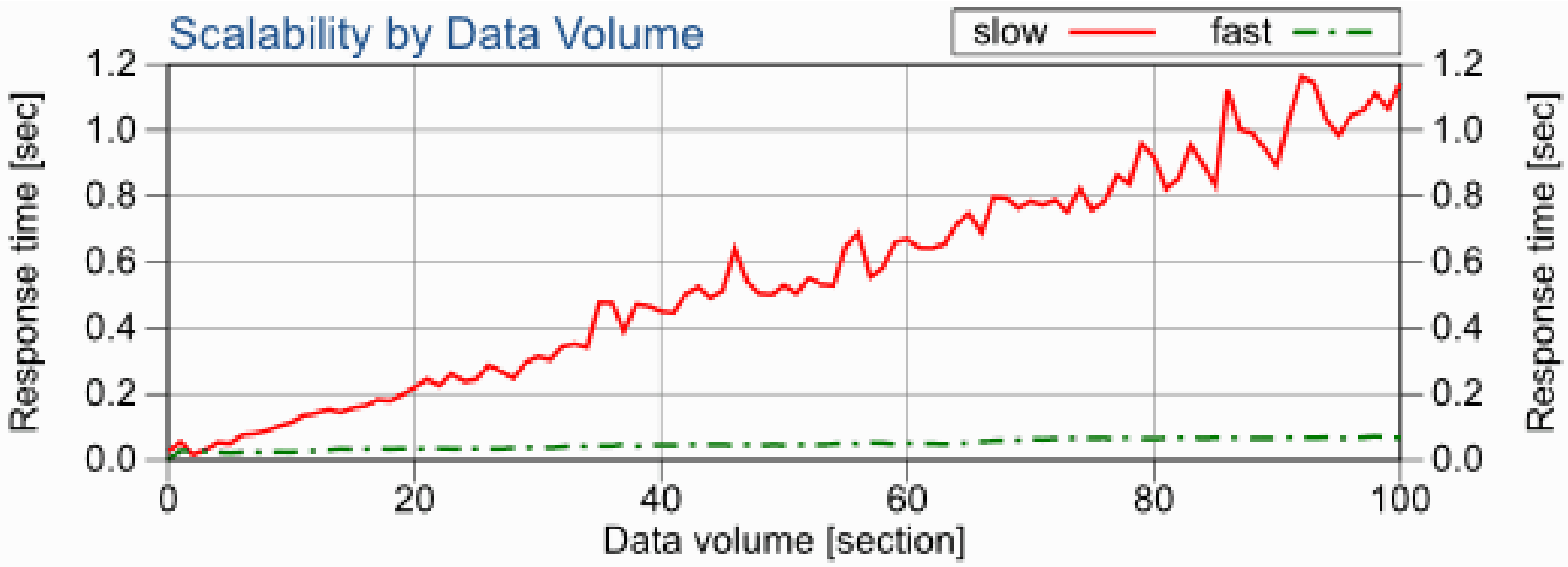
```
SELECT count(*) FROM scale_data
  WHERE section = ? AND id2 = ?
```

- *Section mic* – index1 și apoi index2



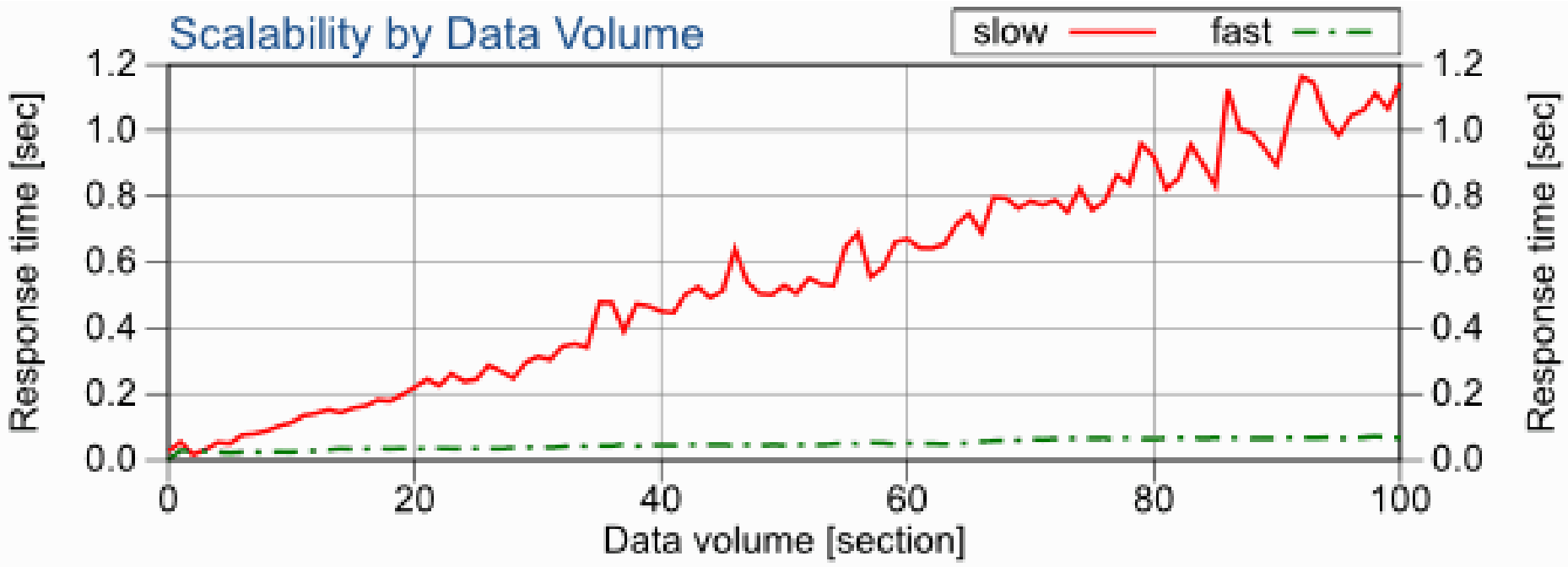
Volumul de date

- **Scalabilitatea** indică dependența performanței în funcție de factori precum **volumul de informații.**



Volumul de date

- index1 – timp dublu față de cel inițial
- index2 – trimp x20 față de cel inițial



Volumul de date

- Răspunsul unei interogări depinde de mai mulți factori. Volumul de date e unul dintre ei.
- Dacă o interogare merge bine în faza de test, nu e neapărat ca ea să funcționeze bine și în producție.
- Care este motivul pentru care apare diferența dintre index1 și index2 ?

Ambele par identice ca execuție:

```
| Id | Operation          | Name        | Rows  | Cost |
-----------------------------------------------------------
|  0 | SELECT STATEMENT   |             |     1 |  972 |
|  1 |  SORT AGGREGATE    |             |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW  |  3000 |  972 |
-----------------------------------------------------------
```

```
-----------------------------------------------------------
| Id   Operation          | Name        | Rows  | Cost |
-----------------------------------------------------------
|  0 | SELECT STATEMENT   |             |     1 |   13 |
|  1 |  SORT AGGREGATE    |             |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST  |  3000 |   13 |
-----------------------------------------------------------
```

Volumul de date

- Ce influențează un index ?
  - table acces
  - scanarea unui interval mare
- Nici unul din planuri nu indică acces pe baza indexului (`TABLE ACCES BY INDEX ROW ID`)
- Unul din intervale este mai mare atunci cand e parcurs…. trebuie să avem acces la "*predicate information*" ca să vedem de ce:

```
---------------------------------------------------------
| Id | Operation          | Name        | Rows  | Cost |
---------------------------------------------------------
|  0 | SELECT STATEMENT   |             |     1 |  972 |
|  1 |  SORT AGGREGATE    |             |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW  |  3000 |  972 |
---------------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A))
       filter("ID2"=TO_NUMBER(:B))
```

```
-----------------------------------------------------
| Id   Operation          | Name        | Rows   | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |             |      1 |   13 |
|  1 |  SORT AGGREGATE    |             |      1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST  |   3000 |   13 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A) AND "ID2"=TO_NUMBER(:B))
```

Volumul de date

- Puteți spune cum a fost construit indexul având planurile de execuție ?

Volumul de date

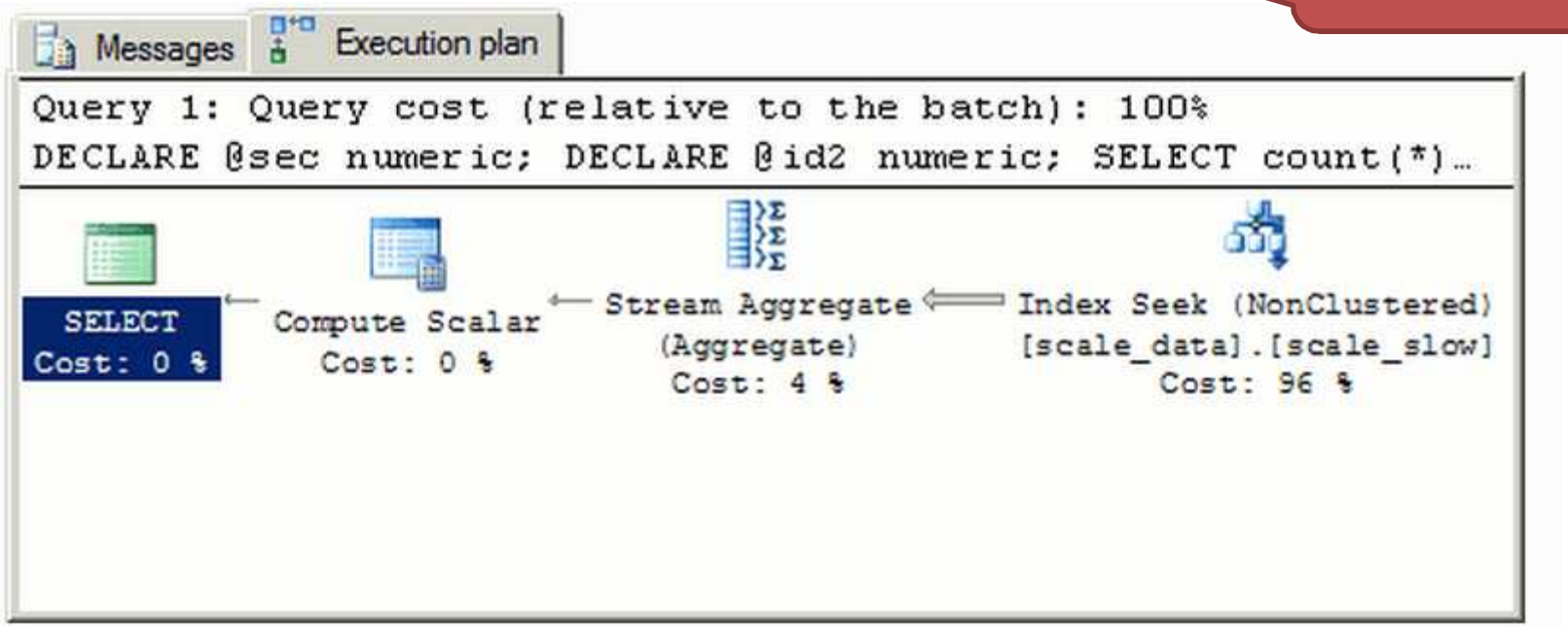
- Puteți spune cum a fost construit indexul având planurile de execuție ?
- `CREATE INDEX scale_slow ON scale_data (section, id1, id2);`
- `CREATE INDEX scale_fast ON scale_data (section, id2, id1);`

Campul id1 este adaugat doar pentru a pastra aceeasi dimensiune (sa nu se creada ca indexul *scale_fast* e mai rapid pentru ca are mai putine campuri in el).

Încărcarea sistemului

- Faptul ca am definit un index pe care îl considerăm bun pentru interogările noastre nu îl face să fie neapărat folosit de QO.
- *SQL Server Management Studio*

Arata predicatul doar ca un tooltip

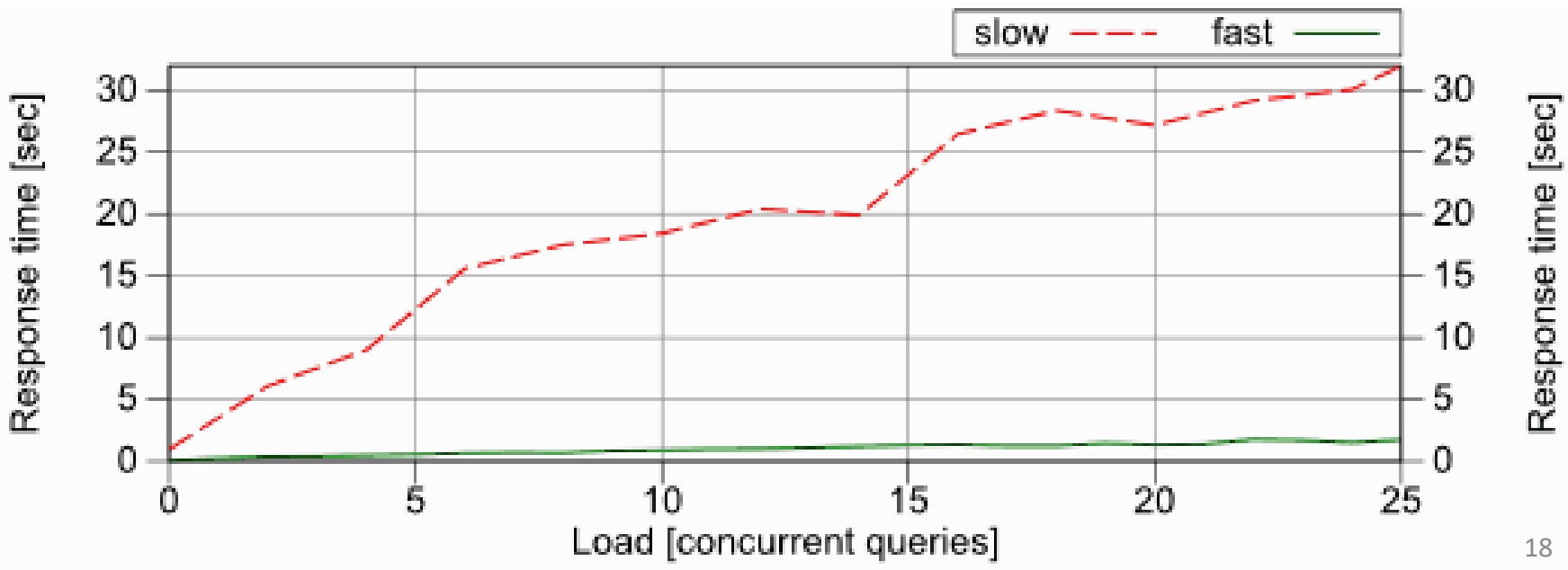


Încărcarea sistemului

- De regulă, împreună cu numărul de înregistrări, crește și numărul de accesări.
- **Numărul de accesari** este alt parametru ce intră în calculul scalabilității.

Încărcarea sistemului

- Dacă inițial era doar o singură accesare, considerând același scenariu dar cu 1-25 interogări concurente, timpul de raspuns crește:



Încărcarea sistemului

- Asta înseamnă că și dacă avem toata baza de date din producție și testăm totul pe ea, tot sunt șanse ca în realitate, din cauza numărului mare de interogări, să meargă mult mai greu.

- **Notă:** atenția dată planului de execuție este mai importantă decât *benchamarkuri* superficiale *(* gen *SQL Server Management Studio)*.

Încărcarea sistemului

- Ne-am putea aștepta ca **hardwareul mai puternic din producție să ducă mai bine sistemul**. În fapt, în faza de development nu există deloc latență – ceea ce nu se întâmplă în producție (unde accesul poate fi întârziat din cauza rețelei).
- http://blog.fatalmind.com/2009/12/22/latency-security-vs-performance/
- http://jamesgolick.com/2010/10/27/we-are-experiencing-too-much-load-lets-add-a-new-server..html

Timpi de răspuns + *throughput*

- Hardware mai performant nu este mai rapid doar poate duce mai multa încărcare. highway (daca adaug 10 benzi, nu inseamna ca si masinile vor merge de 10 ori mai rapid)
- Procesoarele single-core vs procesoarele multi-core (când e vorba de un singur task).
- Scalarea pe orizontală (adăugarea de procesoare) are același efect.
- Pentru a îmbunătăți timpul de răspuns este necesar un arbore eficient (chiar și în NoSQL).

Timpi de răspuns

- Indexarea corectă fac căutarea într-un B-tree în timp logaritmic.
- Sistemele bazate pe NoSQL par să fi rezolvat problema performanței prin scalare pe orizontală [analogie cu indecșii parțiali în care fiecare partiție este stocată pe o mașina diferită].
- Această scalabilitate este totuși limitată la operațiile de scriere într-un model denumit "eventual consistency" [Consistency / Availability / Partition tolerance = CAP theorem] http://en.wikipedia.org/wiki/CAP_theorem

Timpi de răspuns

- Mai mult hardware de obicei nu îmbunătățește sistemul.
- *Latency* al HDD [problema apare cand datele sunt culese din locații diferite ale HDDului – de exemplu în cadrul unei operații JOIN]. SSD?

"Facts"

- *Performance has two dimensions: response time and throughput.*
- *More hardware will typically not improve query response time.*
- *Proper indexing is the best way to improve query response time.*

Join

Join

An SQL query walks into a bar and sees two tables.
He walks up to them and asks 'Can I join you?'

— Source: Unknown

- *Join-ul* transformă datele dintr-un model normalizat într-unul denormalizat care servește unui anumit scop.
- Sensibil la latențe ale discului (și fragmentare).

Join

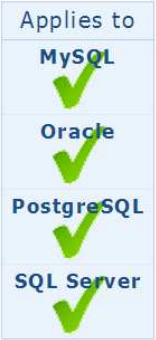
- Reducerea timpilor = indexarea corecta ☺
- Toți algoritmii de join procesează doar două tabele simultan (apoi rezultatul cu a treia, etc).
- Rezultatele de la un join sunt pasate în următoarea operație join fără a fi stocate.
- Ordinea în care se efectuează JOIN-ul influențează viteza de răspuns.[10, 30, 5, 60]
- QO încearcă toate permutările de JOIN.
- Cu cât sunt mai multe tabele, cu atăt mai multe planuri de evaluat. [câte ?]

Join

- Cu cât sunt mai multe tabele, cu atât mai multe planuri de evaluat = n!
- Nu este o problemă când sunt utilizați parametri dinamici [De ce ?]

- Nested Loops
- Hash Joins
- Merged Joins

Join – Nested Loops (anti patern)



- Ca și cum ar fi două interogări: cea exterioară pentru a obține o serie de rezultate dintr-o tabelă și cea interioară ce preia fiecare rând obținut și apoi informația corespondentă din cea de-a doua tabelă.
- Se pot folosi *Nested Selects* pentru a simula algoritmul de nested loops [latența rețelei, usurința implementării, Object-relational mapping (N+1 selects)].

Studiu de caz…

```
select scholarship from
    fg join students on
    fg.lnamea=students.lname
    where fg.lnamea like 'A%';
```

- Care credeţi că va fi tabela interioară şi care cea exterioară ?

Studiu de caz…

```
select scholarship from
    fg join students on
    fg.lnamea=students.lname
    where fg.lnamea like 'A%';
```

- fg – cea exterioară pentru că “speră” ca după filtrare să aibă puține rânduri (cu lname like ‘A%’)
- students – cea interioară

Ce alege să facă când nu sunt indecşi ?

```
---------------------------------------------------------------------------
| Id  | Operation           | Name     | Rows  | Bytes | Cost (%CPU)|
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT    |          |   283 |  8773 | 13458   (1)|
|*  1 |  HASH JOIN          |          |   283 |  8773 | 13458   (1)|
|*  2 |   TABLE ACCESS FULL| STUDENTS |    27 |   594 |     5   (0)|
|*  3 |   TABLE ACCESS FULL| FG       |   544 |  4896 | 13452   (1)|
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
   2 - filter("STUDENTS"."LNAME" LIKE 'A%')
   3 - filter("FG"."LNAMEA" LIKE 'A%')
```

Deci va face hash join (bine, in acest caz)….

Putem forţa tipul joinului:

```
select /* hint */scholarship from
   fg join students on
   fg.lnamea=students.lname
   where fg.lnamea like 'A%';
```

- Hint:

+ USE_NL(students fg) - nested loops

+ use_hash – hash joins

+ use_merge(fg,students) parallel(fg, 4) parallel(students, 4) – merge joins

Nested loops forţat (iese mai rău):

```
-----------------------------------------------------------------------
| Id  | Operation           | Name     | Rows  | Bytes | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT    |          |   283 |  8773 | 15375   (1)|
|   1 |  NESTED LOOPS       |          |   283 |  8773 | 15375   (1)|
|*  2 |   TABLE ACCESS FULL| FG       |   544 |  4896 | 13452   (1)|
|*  3 |   TABLE ACCESS FULL| STUDENTS |     1 |    22 |     4   (0)|
-----------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   2 - filter("FG"."LNAMEA" LIKE 'A%')
   3 - filter("STUDENTS"."LNAME" LIKE 'A%' AND
              "FG"."LNAMEA"="STUDENTS"."LNAME")
```

INTERESANT: filtrul LIKE 'A%' a fost pus peste fg.lnamea – puteți observa cum îl aplică și în students (când îi caută pe cei care se "potrivesc")

Cum ați construi indecșii ?

- Ştiind cum funcţionează nested loops, indexul firesc este în tabela din LOOP-ul interior, pe câmpul peste care se face join-ul.
- Din cauză că students era tabela interioară, să facem indexul peste students.lname:

```
create index idx_students_lname on
  students(lname);
(de copiat pe tabla)
```

Nested loops forţat (neimpresionant):

```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name                | Rows  | Bytes | Cost (%
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                     |   283 |  8773 | 14540
|   1 |  NESTED LOOPS                |                     |       |       |
|   2 |   NESTED LOOPS               |                     |   283 |  8773 | 14540
|*  3 |    TABLE ACCESS FULL         | FG                  |   544 |  4896 | 13452
|*  4 |    INDEX RANGE SCAN          | IDX_STUDENTS_LNAME  |     1 |       |     1
|   5 |   TABLE ACCESS BY INDEX ROWID| STUDENTS            |     1 |    22 |     2

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - filter("FG"."LNAMEA" LIKE 'A%')
   4 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
       filter("STUDENTS"."LNAME" LIKE 'A%')
```

Se foloseste indexul ca sa gaseasca studentii – si se vede ca se face access pe criteriul de join; filtrul este peste like… [costul totusi e 14.5k ☹]

```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name                | Rows  | Bytes | Cost (%
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                     |   283 |  8773 | 14540
|   1 |  NESTED LOOPS                |                     |       |       |
|   2 |   NESTED LOOPS               |                     |   283 |  8773 | 14540
|*  3 |    TABLE ACCESS FULL         | FG                  |   544 |  4896 | 13452
|*  4 |    INDEX RANGE SCAN          | IDX_STUDENTS_LNAME  |     1 |       |     1
|   5 |   TABLE ACCESS BY INDEX ROWID| STUDENTS            |     1 |    22 |     2

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - filter("FG"."LNAMEA" LIKE 'A%')
   4 - access("FG"."LNAMEA"="STUDENTS"."
       filter("STUDENTS"."LNAME" LIKE 'A
```

BUG ?!

Estimare

QO doar face o estimare a ceea ce se intampla… daca ar face efectiv rularea queryului sa obtina rezultatul real, ar pierde mult timp (si nu ar mai fi “estimare” :D)

```
SQL> select count(*) from fg where lnamea like 'A%';

  COUNT(*)
----------
    133504
```

Realitate

In fapt l-ar ajuta foarte mult ca din cele 5M linii din fg sa ajunga rapid la cele 133k pe care le doreste.

Atunci când avem nested loops:

- Indexați criteriul după care se face joinul din tabela interioară. Dacă veți indexa și ceva în tabela exterioară (pentru a face mai rapid accesul la liniile dorite, va fi perfect).
- În cazul de față ar fi interesant să facem acces acolo unde este lnamea.
- Se poate schimba ordinea tabelelor la faza asta… cea interioară să devină exterioară…

Să-l ajutăm aşadar….

```
create index idx_join_fg
                    on fg(lnamea);
```

(de copiat pe tabla)

```
---------------------------------------------------------------------------
| Id  | Operation                    | Name                | Rows  | Bytes | Cost (%
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                     |   283 |  8773 |  1025
|   1 |  NESTED LOOPS                |                     |   283 |  8773 |  1025
|   2 |   TABLE ACCESS BY INDEX ROWID| STUDENTS            |    27 |   594 |     4
|*  3 |    INDEX RANGE SCAN          | IDX_STUDENTS_LANME  |     9 |       |     2
|*  4 |   INDEX RANGE SCAN           | IDX_JOIN_FG         |    10 |    90 |    38
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("STUDENTS"."LNAME" LIKE 'A%')
       filter("STUDENTS"."LNAME" LIKE 'A%')
   4 - access("STUDENTS"."LNAME"="FG"."LNAMEA")
       filter("FG"."LNAMEA" LIKE 'A%')
```

Atunci când avem nested loops:

- Dacă veţi indexa criteriul exterior (dinafara joinului), aveţi şanse să scădeţi foarte mult timpul operaţiei join.
- HOWEVER, când sunt utilizate joinurile de tipul nested loops, intotdeauna puneţi index pe criteriul de selecţie din tabela interioară (de fapt dacă găsește index, o va folosi drept interioară).

Join – Cu *doctrine* putem vedea ce se intâmpla de fapt la nivel de cereri BD:

Doctrine

Only on source code level—don't forget to disable this for production. Consider building your own configurable logger.

```
$logger = new \Doctrine\DBAL\Logging\EchoSqlLogger;
$config->setSQLLogger($logger);
```

Join – Ce fac unii “programatori”…

```
$qb = $em->createQueryBuilder();
$qb->select('e')
   ->from('Employees', 'e')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();

foreach ($r as $row) {
   // process Employee
   foreach ($row->getSales() as $sale) {
      // process Sale for Employee
   }
}
```

Join – Ce se întâmplă în spate…

Doctrine 2.0.5 generates N+1 `select` queries:

```
SELECT e0_.employee_id AS employee_id0 -- MORE COLUMNS
  FROM employees e0_
 WHERE UPPER(e0_.last_name) LIKE ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?
```

PRO TIP

- Dacă aveţi de făcut nested loops, nu o faceţi în aplicaţia PHP / Java / orice altceva… Făceţi-o direct în serverul de baze de date:

- în BD nu avem latența din rețea;
- în BD nu sunt transferate datele intermediare (care sunt *piped* în BD);
- ORM-urile de obicei nu fac JOINurile în BD ☹ (ci le fac prin nested loops în interiorul app).

Join – nested selects

- *eager fetching* – poate face un join automat pentru a-mi da direct toate informațiile (denormalizare)
- Totuși *eager fetching* nu este bun atunci când este nevoie doar de informații dintr-o tabelă.
- O configurare statică (în genul celor *eager*) nu este mereu o soluție bună.
- Cele mai multe ORM permit SQL joins:

```
$qb = $em->createQueryBuilder();
$qb->select('e,s')
   ->from('Employees', 'e')
   ->leftJoin('e.sales', 's')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();
```

Doctrine 2.0.5 generates the following SQL statement:

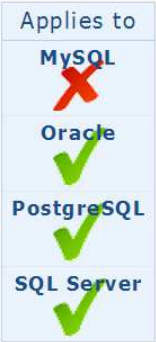
```
SELECT e0_.employee_id AS employee_id0
       -- MORE COLUMNS
  FROM employees e0_
  LEFT JOIN sales s1_
         ON e0_.subsidiary_id = s1_.subsidiary_id
        AND e0_.employee_id = s1_.employee_id
 WHERE UPPER(e0_.last_name) LIKE ?
```

Join – nested selects

- Sunt bune dacă sunt întoarse un număr mic de înregistrări.

- http://blog.fatalmind.com/2009/12/22/latency-security-vs-performance/

Join – Hash join



- Evită traversarea multiplă a B-tree din cadrul inner-querry (din nested loops) construind câte un hash pentru înregistrările candidat.
- Dacă una din tabele este foarte mare, cealaltă este incărcată în memorie, pe baza ei se construiește un hash având ca și chei câmpurile utilizate în join și ca valori restul rândului.
- *Hash join* îmbunătățit dacă sunt selectate mai puține coloane.
- A se indexa predicatele independente din where pentru a îmbunătăți eficiența.

Având indecșii construiti anterior…

```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name               | Rows  | Bytes | Cost (%C
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                    |   283 |  8773 |   127
|*  1 |  HASH JOIN                   |                    |   283 |  8773 |   127
|   2 |   TABLE ACCESS BY INDEX ROWID| STUDENTS           |    27 |   594 |     4
|*  3 |    INDEX RANGE SCAN          | IDX_STUDENTS_LANME |     9 |       |     2
|*  4 |   INDEX RANGE SCAN           | IDX_JOIN_FG        |   544 |  4896 |   122
-----------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("STUDENTS"."LNAME"="FG"."LNAMEA")
   3 - access("STUDENTS"."LNAME" LIKE 'A%')
       filter("STUDENTS"."LNAME" LIKE 'A%')
   4 - access("FG"."LNAMEA" LIKE 'A%')
       filter("FG"."LNAMEA" LIKE 'A%')
```

Am putea crede ca ambii indecsi participa “puternic” la realizarea JOIN-ului…
De fapt daca scapam de IDX_STUDENTS_LANME (sorry for typo)…

Doar IDX_JOIN_FG (peste lnameA):

```
-----------------------------------------------------------------------
| Id  | Operation          | Name        | Rows  | Bytes | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT   |             |   283 |  8773 |   128   (1)|
|*  1 |  HASH JOIN         |             |   283 |  8773 |   128   (1)|
|*  2 |   TABLE ACCESS FULL| STUDENTS    |    27 |   594 |     5   (0)|
|*  3 |   INDEX RANGE SCAN | IDX_JOIN_FG |   544 |  4896 |   122   (0)|
-----------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("STUDENTS"."LNAME"="FG"."LNAMEA")
   2 - filter("STUDENTS"."LNAME" LIKE 'A%')
   3 - access("FG"."LNAMEA" LIKE 'A%')
       filter("FG"."LNAMEA" LIKE 'A%')
```

Costul a crescut doar cu (era 127 si acum este 128). Deci de fapt eficienta vine din Indexarea conditiilor dinafara joinului.

Join – Hash join

- **Indexarea predicatelor utilizate în join nu îmbunătățesc eficiența hash join !!!**
- Un index ce ar putea fi utilizat este peste fg.lname (care este în clauza where)

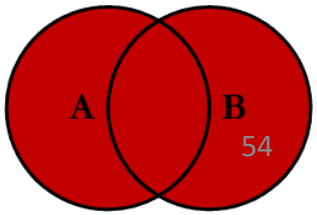
Join – Hash join

- Ordinea condițiilor din join nu influențează viteza deoarece oricum QO va încărca în memorie tabela mai mică și îi va face hash-ul (la nested loops ar fi influențat).

Join – Sort Merge

Applies to
MySQL ✗
Oracle ✓
PostgreSQL ✓
SQL Server ✓

- Combină două tabele preluând date fie dintr-unul, fie din celălalt, fie din ambele.
- Aceiași indecși ca și la hash-join (adică doar pentru condițiile separate, nu și pentru cele din join).
- Ordinea joinurilor nu contează.
- Algoritm foarte util pentru outer joins (în care sunt întoarse înregistrările care fac match în ambele tabele) dar și pt L/R joins.



```
---------------------------------------------------------------------------
| Id  | Operation           | Name        | Rows  | Bytes | Cost (%CPU)|
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT    |             |   283 |  8773 |   128   (1)|
|   1 |  MERGE JOIN         |             |   283 |  8773 |   128   (1)|
|*  2 |   INDEX RANGE SCAN  | IDX_JOIN_FG |   544 |  4896 |   122   (0)|
|*  3 |   SORT JOIN         |             |    27 |   594 |     6  (17)|
|*  4 |    TABLE ACCESS FULL| STUDENTS    |    27 |   594 |     5   (0)|
---------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("FG"."LNAMEA" LIKE 'A%')
       filter("FG"."LNAMEA" LIKE 'A%')
   3 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
       filter("FG"."LNAMEA"="STUDENTS"."LNAME")
   4 - filter("STUDENTS"."LNAME" LIKE 'A%')
```

Şi aşa ar arăta fără nici un index:

```
-----------------------------------------------------------------------
| Id  | Operation           | Name     | Rows  | Bytes | Cost (%CPU)| Ti
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT    |          |   283 |  8773 | 13459   (1)| 00
|   1 |  MERGE JOIN         |          |   283 |  8773 | 13459   (1)| 00
|   2 |   SORT JOIN         |          |    27 |   594 |     6  (17)| 00
|*  3 |    TABLE ACCESS FULL| STUDENTS |    27 |   594 |     5   (0)| 00
|*  4 |   SORT JOIN         |          |   544 |  4896 | 13453   (1)| 00
|*  5 |    TABLE ACCESS FULL| FG       |   544 |  4896 | 13452   (1)| 00

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - filter("STUDENTS"."LNAME" LIKE 'A%')
   4 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
       filter("FG"."LNAMEA"="STUDENTS"."LNAME")
   5 - filter("FG"."LNAMEA" LIKE 'A%')
```

Join – Sort Merge

Smith	40
Winand	44
Tyler	46

40	EUR 100
40	EUR 250
42	EUR 100
44	EUR 500

FULL OUTER JOIN using sort-merge

Și un plan mai “exotic”

- Pentru:

```
select /*+ use_merge(fg,students)
  parallel(fg, 4) parallel(students,
  4) */ scholarship from
  fg full outer join students
  on fg.lnamea=students.lname
  where fg.lnamea like 'A%';
```

```
-----------------------------------------------------------------------------
| Id  | Operation                | Name     | Rows  | Bytes | Cost (%CPU)| Time
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |  1569 | 20397 | 26917   (1)| 00:05
|   1 |  VIEW                    |          |  1569 | 20397 | 26917   (1)| 00:05
|   2 |   UNION-ALL              |          |       |       |            |
|   3 |    MERGE JOIN OUTER      |          |   544 | 16864 | 13459   (1)| 00:02
|   4 |     SORT JOIN            |          |   544 |  4896 | 13453   (1)| 00:02
|*  5 |      TABLE ACCESS FULL| FG       |   544 |  4896 | 13452   (1)| 00:02
|*  6 |     SORT JOIN            |          |    27 |   594 |     6  (17)| 00:00
|*  7 |      TABLE ACCESS FULL| STUDENTS |    27 |   594 |     5   (0)| 00:00
|*  8 |    FILTER                |          |       |       |            |
|*  9 |     HASH JOIN ANTI       |          |  1025 | 31775 | 13458   (1)| 00:02
|  10 |      TABLE ACCESS FULL| STUDENTS |  1025 | 22550 |     5   (0)| 00:00
|  11 |      TABLE ACCESS FULL| FG       |  4842K|    41M| 13439   (1)| 00:02
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   5 - filter("FG"."LNAMEA" LIKE 'A%')
   6 - access("FG"."LNAMEA"="STUDENTS"."LNAME"(+))
       filter("FG"."LNAMEA"="STUDENTS"."LNAME"(+))
   7 - filter("STUDENTS"."LNAME"(+) LIKE 'A%')
   8 - filter(NULL LIKE 'A%')
   9 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
```

```
---------------------------------------------------------------------------------------
| Id  | Operation                        | Name     | Rows  | Bytes | Cost (%CPU)| Time
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                 |          |  1569 | 20397 |  2183   (1)| 00:0
|   1 |  VIEW                            |          |  1569 | 20397 |  2183   (1)| 00:0
|   2 |   UNION-ALL                      |          |       |       |            |
|   3 |    MERGE JOIN OUTER              |          |   544 | 16864 |   127   (1)| 00:0
|*  4 |     INDEX RANGE SCAN             | IDX2     |   544 |  4896 |   122   (0)| 00:0
|*  5 |     SORT JOIN                    |          |    27 |   594 |     5  (20)| 00:0
|   6 |      TABLE ACCESS BY INDEX ROWID| STUDENTS |    27 |   594 |     4   (0)| 00:0
|*  7 |       INDEX RANGE SCAN           | IDX1     |     9 |       |     2   (0)| 00:0
|*  8 |    FILTER                        |          |       |       |            |
|   9 |     NESTED LOOPS ANTI            |          |  1025 | 31775 |  2055   (0)| 00:0
|  10 |      TABLE ACCESS FULL           | STUDENTS |  1025 | 22550 |     5   (0)| 00:0
|* 11 |      INDEX RANGE SCAN            | IDX2     |     1 |     9 |     2   (0)| 00:0
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   4 - access("FG"."LNAMEA" LIKE 'A%')
       filter("FG"."LNAMEA" LIKE 'A%')
   5 - access("FG"."LNAMEA"="STUDENTS"."LNAME"(+))
       filter("FG"."LNAMEA"="STUDENTS"."LNAME"(+))
   7 - access("STUDENTS"."LNAME"(+) LIKE 'A%')
       filter("STUDENTS"."LNAME"(+) LIKE 'A%')
   8 - filter(NULL LIKE 'A%')
  11 - access("FG"."LNAMEA"="STUDENTS"."LNAME")
```

Aici au fost indexate campurile din criteriul de join (dar de fapt lnamea like ‘A%’ e cel folosit…).

Join – Sort Merge

- Destul de greoi de utilizat din cauză că ambele tabele trebuie să fie sortate după câmpul utilizat în join.

Clustering Data

Clustering Data

- Un cluster = serie de obiecte (de obicei de aceeași natură) ce apar împreună.
- Un cluster de calculatoare este o grupare de calculatoare care “se ajută” simultan.
- Utilizate pentru

  - a rezolva o problema complexa (*high performance clusters*)

  - a mari disponibilitatea (*failover cluster*)

Clustering Data

- În cazul calculatoarelor mai există un tip de cluster: Data Cluster
- Clustering data = așezarea datelor ce au probabilitatea imediată de acces unele lânga altele (de exemplu atunci când defragmentăm HDD-ul, facem acest tip de clusterizare).
- Computer cluster este destul de întâlnit și între cele două se face confuzie (“Let’s use a cluster to improve DB performance”)

Clustering Data

1. Cel mai simplu tip de cluster este rândul: bazele de date **stochează toate coloanele dintr-un rând în același block** (DB block) dacă e posibil.

Column Stores

Column oriented databases*, or column-stores, organize tables in a columned way. This model is beneficial when accessing many rows but only a few columns—a pattern that is very common in data warehouses (OLAP).*

Clustering Data

2. Indecșii construiesc clustere de rânduri – frunzele B-tree-ului **stochează coloanele într-o maniera ordonată** (valorile consecutive sunt puse unele lângă altele).

Si cum de obicei avem nevoie de valori apropiate, le putem prelua mai multe dintr-o data.

- **Indecșii construiesc clustere de rânduri cu valori similare** = *second power of indexing.*

Clustering Data- Index Filter Predicates Used Intentionally

- Când apare "*Index filter*" de multe ori înseamnă că indexul nu a fost construit corect și că probabil există o variantă mai eficientă (în care ceea ce este filtrat este folosit de index).
- *Index filtering* poate fi folosit și într-un scop bun: pentru gruparea datelor ce sunt accesate consecutiv.

Clustering Data- Index Filter Predicates Used Intentionally

- Clauzele WHERE ce nu pot fi utilizate ca și predicate de acces sunt cele mai bune pentru această tehnică:

```
SELECT lnameA, fnameA, dobA FROM fg
WHERE valueA = ?
    AND UPPER(lnameA) LIKE '%ACK%';
```

Indexul e inutil indiferent dacă e după **`lnameA`** sau după **`upper(lnameA)`**. Dupa ce indexăm ?

No index…

```
-----------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |  2421 |   113K| 13454   (1)| 00:02:42
|*  1 |  TABLE ACCESS FULL| FG   |  2421 |   113K| 13454   (1)| 00:02:42
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("VALUEA"=9 AND UPPER("LNAMEA") LIKE '%ACK%')
```

Clustering Data- Index Filter Predicates Used Intentionally

- *Tabble acces* nu este neapărat o povară dacă înregistrările ar fi stocate grupat în același block și BD ar putea să le returneze într-o singură operație de tip *read*.
- Dacă rândurile sunt dispersate, BD va trebui să citească din fiecare block în parte.
- Eficiența depinde de distribuția fizică a datelor.

Index on fg(valueA)

```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name       | Rows  | Bytes | Cost (%CP
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |            |  2421 |   113K|  2317   (
|*  1 |  TABLE ACCESS BY INDEX ROWID| FG         |  2421 |   113K|  2317   (
|*  2 |   INDEX RANGE SCAN           | IDX_VALUEA | 19369 |       |  1439   (
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(UPPER("LNAMEA") LIKE '%ACK%')
   2 - access("VALUEA"=9)
```

Avantajul este ca le gaseste pe toate cu valoarea 9 “dintr-un foc”.

Clustering Data- Index Filter Predicates Used Intentionally

- Corelația dintre ordinea din index și cea din tabelă se numește ***index clustering factor.***

  (nr blocuri < ICF < nr rânduri)

- ICF = dacă am un număr de rânduri ce pot fi stocate în nr blocuri, atunci putem sa le citim din nr randuri accesari la HDD (dacă nu găsim niciodată două consecutive în același bloc) sau din nr blocuri (dacă la o citire toate sunt preluate corect și în ordinea dorită).

Clustering Data- Index Filter Predicates Used Intentionally

- Corelația dintre ordinea din index și cea din tabelă se numește ***index clustering factor.***

  (nr blocuri < ICF < nr rânduri)

- Este posibilă îmbunătățirea eficienței re-aranjând rândurile din tabel (ce se întâmplă dacă avem doi indecși diferiți ?)
- Chiar dacă am face ordonarea după un singur index, și așa e complicat pentru că BD nu permite decât un acces rudimentar la această ordine (*row sequencing*).

Clustering Data- Index Filter Predicates Used Intentionally

- Aici apare partea cu clusterizarea: utilizarea unui index din două coloane pentru a fi stocate într-o ordine bine definită.
- Se va extinde indexul pentru a acoperi chiar și coloana “neindexabilă”:

Index on fg(valueA, Upper(lnameA))

```
-----------------------------------------------------------------------------------
| Id  | Operation                    | Name                  | Rows  | Bytes | Cost (%C
-----------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                       | 31187 |  1735K|   109
|   1 |  TABLE ACCESS BY INDEX ROWID| FG                    | 31187 |  1735K|   109
|*  2 |   INDEX RANGE SCAN           | IDX_VALUEA_UPPLNAMEA  |   968 |       |    64
-----------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("VALUEA"=9)
       filter(UPPER("LNAMEA") LIKE '%ACK%')
```

De ce este mai bine așa ?

Clustering Data- Index Filter Predicates Used Intentionally

- Se observă că filtrul este direct aplicat în pasul 2.
- De data aceasta pasul 2 returnează doar 968 rânduri (în cazul anterior erau 19369).

Clustering Data- Index Filter Predicates Used Intentionally

- Această abordare ignoră relevanța ordinii coloanelor. De obicei, partea ce va fi filtrată este bine să fie pusă ultima (pentru ca indexul să meargă corect pe primele coloane – adică să facă acces).
- Nu adăugați toate coloanele în index

Clustering Data- Index-Only Scan

- Una dintre cele mai puternice metode de "*tuning*".
- Nu numai că evită evaluarea clauzei *where* dar evită accesarea tabelei atunci când coloanele selectate se găsesc chiar în index.

Clustering Data- Index-Only Scan

```
create index idx_only_scan on fg(lnameA,
  fnameA, dobA);

select fnameA, lnameA, dobA from fg where
  lnameA = 'Jackson';
```

```
---------------------------------------------------------------------
| Id  | Operation        | Name          | Rows  | Bytes | Cost (%CPU)|
---------------------------------------------------------------------
|   0 | SELECT STATEMENT |               |   544 | 19040 |   110   (0)|
|*  1 |  INDEX RANGE SCAN| IDX_ONLY_SCAN |   544 | 19040 |   110   (0)|
---------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("LNAMEA"='Jackson')
```

Clustering Data- Index-Only Scan

- Indexul acoperă întreaga interogare și din acest motiv se mai numește "**covering index**" (poate fi identificat prin faptul că nu este accesată deloc tabela ci doar indexul).
- Numărul mare de linii selectate poate avea un impact negativ asupra eficienței (mai ales dacă nu sunt în același cluster – de exemplu dacă aș căuta și după un anumit dob ar putea merge mai greu decât dacă aș căuta numai după lnamea și fnamea).

```
plan for select fnameA, lnameA, dobA from fg where lnameA = 'Jackson' and fnamea='Michael';
-----------------------------------------------------------------------------
| Id  | Operation        | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT |               |   484 | 16940 |     3   (0)| 00:00:01 |
|*  1 |  INDEX RANGE SCAN| IDX_ONLY_SCAN |   484 | 16940 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("LNAMEA"='Jackson' AND "FNAMEA"='Michael')
```

```
plan for select fnameA, lnameA, dobA from fg where lnameA = 'Jackson' and dobA=sysdate;
-----------------------------------------------------------------------------
| Id  | Operation        | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT |               |   484 | 16940 |   110   (0)| 00:00:02 |
|*  1 |  INDEX RANGE SCAN| IDX_ONLY_SCAN |   484 | 16940 |   110   (0)| 00:00:02 |
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("LNAMEA"='Jackson' AND "DOBA"=SYSDATE@!)
       filter("DOBA"=SYSDATE@!)
```

Clustering Data- Index-Only Scan

- Index-only este o strategie agresivă.
- **Nu proiectați indexul în acest scop** pentru că e ca și cum ați copia tabela cu bucățile care vă interesează – ocupă spațiu și timp pentru operații *insert / edit / delete* (reechilibrarea B-tree).

Clustering Data- Index-Only Scan

- Index-only scan poate cauza suprize neplăcute

```
select fnameA, lnameA, dobA from fg
  where lnameA = 'Jackson' AND valueA
  between 4 and 10;
```

- Ce se întâmplă ?

```
---------------------------------------------------------------------------
| Id  | Operation                    | Name        | Rows  | Bytes | Cost (%CPU)|
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |             |   121 |  5808 |   239   (0)|
|*  1 |  TABLE ACCESS BY INDEX ROWID| FG          |   121 |  5808 |   239   (0)|
|*  2 |   INDEX RANGE SCAN           | IDX_JOIN_FG | 19369 |       |    38   (0)|
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("VALUEA">=4 AND "VALUEA"<=10)
   2 - access("LNAMEA"='Jackson')
```

- Din cauza că se face acces în tabelă, costul a crescut. Accesarea oricărei coloane care nu este în index va avea acest efect.

Clustering Data- Index-Only Scan

- Scrieţi comentarii când folositi selecturi de tipul Index-Only scan pentru a vă asigura că nu adăugaţi din greşeala noi predicate ce vor împiedica accesul în stilul index-only scan.
- Scrieţi comentarii când doriţi să faceţi selecturi cu index only scan (în php/java/etc)

Clustering Data- Index-Only Scan

- Altă problemă poate să apară de la FBI.
- UPPER(fnameA) nu poate fi folosit în Index Only Scan dacă se doreşte selectarea coloanei fnameA (pentru că indexul ţine minte valoarea funcţiei şi nu valoarea coloanei) – încercaţi să indexaţi valoarea originală dacă ştiţi că o veţi folosi tot pe ea.
- Nu se va aplica o funcţie pe un câmp din index.

Clustering Data- Index-Only Scan

- Mereu selectaţi doar coloanele de care aveţi nevoie. Dacă selectaţi de genul “select * …” în nici un caz nu veţi folosi Index-Only Scan.
- Există anumite limitări în ceea ce priveşte dimensiunea indexului (deci nu puteți pune tot în index pentru a face numai Index-only Scan):

Limitations of Index-Only Scan

- The maximum index key length depends on the block size and the index storage parameters (75% of the database block size minus some overhead). A B-tree index is limited to 32 columns.
- When using Oracle 11*g* with all defaults in place (8k blocks), the maximum index key length is 6398 bytes. Exceeding this limit causes the error message "ORA-01450: maximum key length (6398) exceeded."

https://docs.oracle.com/cd/B19306_01/server.102/b14231/indexes.htm

Clustering Data- Index-Only Scan

- Interogările care nu selectează nici o coloană din tabel sunt executate cu Index-Only scans.

Puteţi găsi un exemplu ?

Clustering: Index-Organized Tables

- Index-Only Scan execută interogarea SQL doar folosind informaţiile redundante existente în index.
- Dacă am pune toate informaţiile în index, de ce am mai avea nevoie de tabela originală ?

- Într-adevăr, există aceasta abordare: *index-organized tables* (IOT) sau *clustered index*.

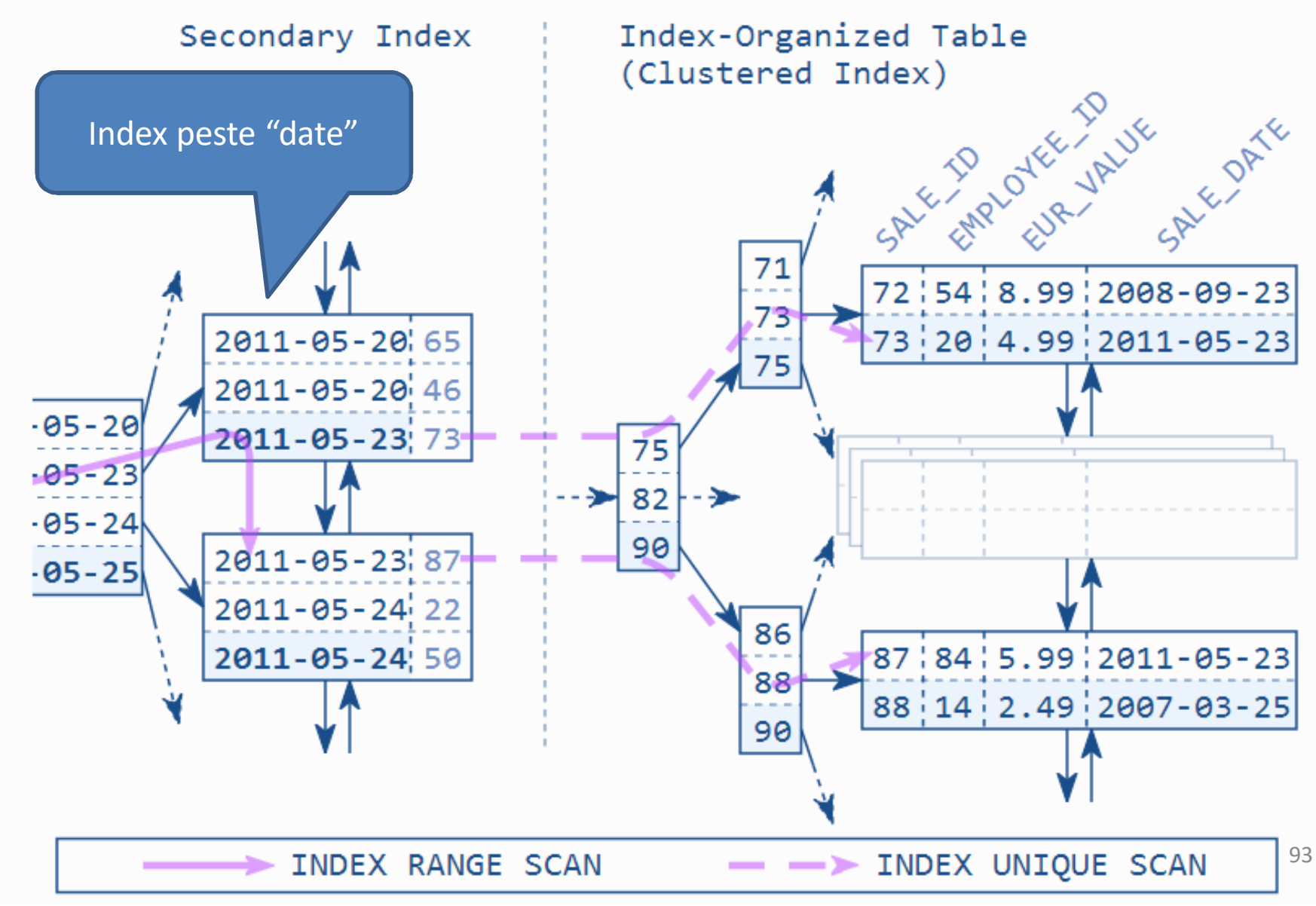
Clustering: Index-Organized Tables

- Avantaje:
  - nu se mai foloseşte tabelul şi se poate renunţa la el;
  - orice acces într-un IOT este un index-only scan.
- Ambele avantaje “sună bine” dar sunt greu de obţinut în practică.

Clustering: Index-Organized Tables

- Problema: când se doreşte un nou index pe aceeaşi tabelă: noul index va pointa prin intermediul unei chei către informaţia logica din indexul original (şi va face index unique scan dupa aceasta cheie).
- Accesarea unui IOT după un al doilea index este foarte ineficientă.

Clustering: Index-Organized Tables



Clustering: Index-Organized Tables

- Al doilea index păstrează clustering keys pentru fiecare înregistrare.
- Am putea de exemplu să interogăm doar ID-urile (şi accesul ar fi de tipul index-scan only):

Clustering: Index-Organized Tables

- Eficienţa este pierdută atunci când este folosit un al doilea index.
- Clustering key este de obicei mai lung decât ROWID (deci şi indecşii secundari vor ocupa mai mult spaţiu).
- Concluzia: DOAR tabelele ce folosesc un singur index sunt cel mai bine să fie implementate ca un IOT.

Clustering: Index-Organized Tables

```
CREATE TABLE (
   id    NUMBER NOT NULL PRIMARY KEY,
   [...]
) ORGANIZATION INDEX;
```

Indecşii în sortare (ORDER BY)

Sortarea

- Sortarea este o acţiune anevoioasa pentru bazele de date, în principal pentru că toate înregistrările trebuie aduse într-un buffer în care să se facă sortarea.
- **Indecşii reţin informaţiile într-o manieră ordonată** – deci putem utiliza un index pentru a uşura munca operaţiilor de tip "*order by*".

Sortarea

- Un INDEX RANGE SCAN care întoarce multe rânduri poate fi ineficient – pentru că timpul necesar pentru row acces ar putea fi mai mare decât dacă s-ar face FULL TABLE SCAN şi apoi s-ar sorta rezultatele.

Sortarea

- Un index construit dupa criteriul ce este utilizat în clauza ORDER BY va evita operaţia de sortare şi va putea întoarce foarte rapid înregistrările **dintr-un anumit interval**.
- Indexul face ca operaţiile de tip ORDER BY să fie executate într-o maniera **pipelined** = **3rd power of index**.
- *Pipelining negates the need to build huge collections by piping rows out of the function as they are created, saving memory and allowing subsequent processing to start before all the rows are generated.*
- Din cauza ca sunt gata sortate in index, nu trebuie sa le mai preia din tabela si sa le sorteze. Poate sa citeasca primul rowid direct din index si sa stie ca e primul, sa se duca apoi in tabela si sa preia restul informatiilor si sa le emită apoi sa mearga pe al doilea rowid etc….

Indexarea pentru ORDER BY

Applies to
MySQL ✓
Oracle ✓
PostgreSQL ✓
SQL Server ✓

- Interogările SQL cu o clauza ORDER BY nu trebuie să sorteze rezultatul dacă indexul deja oferă această ordine:

```
select lnameA, fnameA, dobA
   from fg
   order by dobA;
```

Indexarea pentru ORDER BY

Fără index:

```
-----------------------------------------------------------------------
| Id  | Operation          | Name | Rows  | Bytes |TempSpc| Cost (%
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT   |      |  5102K|   136M|       | 52994
|   1 |  SORT ORDER BY     |      |  5102K|   136M|   195M| 52994
|   2 |   TABLE ACCESS FULL| FG   |  5102K|   136M|       | 13342
-----------------------------------------------------------------------
```

Observaţi că de fapt sortarea face costul să “sară”…

Indexarea pentru ORDER BY

- Pentru a scapa de sortare, ar trebui ca indexul sa fie construit peste criteriul de ordonare:

```
create index idx_dobA on fg(dobA);
```

```
---------------------------------------------------------------------------
| Id  | Operation          | Name | Rows  | Bytes |TempSpc| Cost (%CPU)|
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |      |  5102K|   136M|       | 52994   (1)|
|   1 |  SORT ORDER BY     |      |  5102K|   136M|   195M| 52994   (1)|
|   2 |   TABLE ACCESS FULL| FG   |  5102K|   136M|       | 13342   (1)|
---------------------------------------------------------------------------
```

NU FOLOSESTE INDEXUL… DE CE?

Indexarea pentru ORDER BY

- Doua posibilităţi: interogăm doar dob (ca să nu mai facă table access) sau introducem în index si lnameA, fnameA.

```
create index idx_dobA on
  fg(dobA, lnameA, fnameA);
```

```
-----------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%C
-----------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |  5102K|   136M| 26489
|   1 |  INDEX FULL SCAN  | IDX_DOBA |  5102K|   136M| 26489
-----------------------------------------------------------------
```

Indexarea pentru ORDER BY

- Să încercăm acum un index peste două câmpuri: valueA & dobA.

```
create index idx_vala_doba on fg(valuea,
  doba);
```

Cum va fi executat urmatorul select:

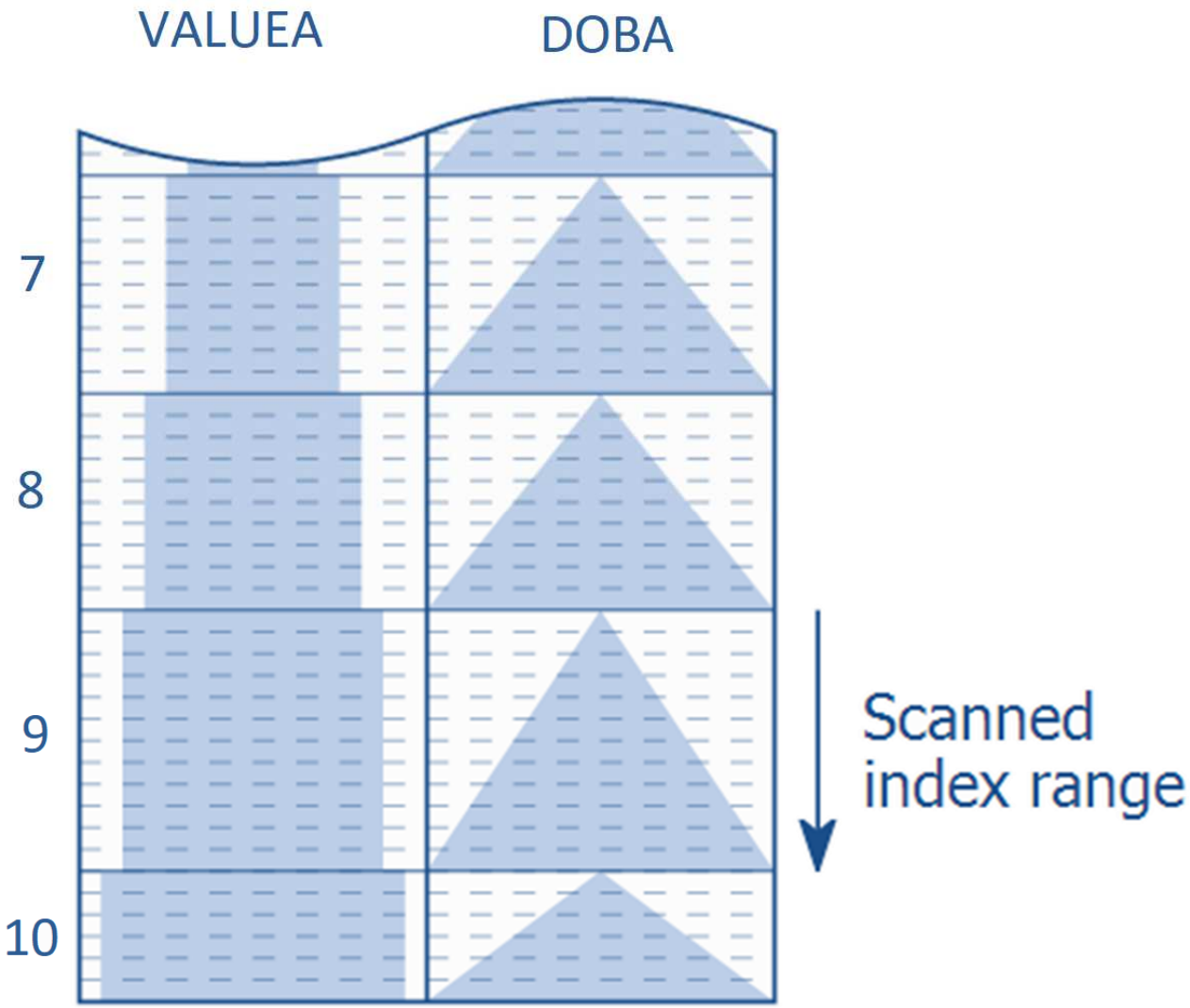
```
select valuea, doba from fg where
  valuea=9 order by doba;
```

?

```
---------------------------------------------------------------------
| Id  | Operation        | Name          | Rows  | Bytes | Cost (%CPU)|
---------------------------------------------------------------------
|   0 | SELECT STATEMENT |               |   784K|  8424K|  2413   (1)|
|*  1 |  INDEX RANGE SCAN| IDX_VALA_DOBA |   784K|  8424K|  2413   (1)|
---------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("VALUEA"=9)
```

Se face access dupa VALUEA si apoi se citeste
Indexul pentru ca datele sunt deja sortate
Dupa câmpul dob.

Indexarea pentru ORDER BY



Listă dublă înlănţuită

- Dacă frunzele nu ar fi dublu înlănţuite, atunci nu am putea ca, utilizând acelaşi index să cerem datele în ordine descrescătoare şi să nu apară sortare… să vedem dacă apare sort:

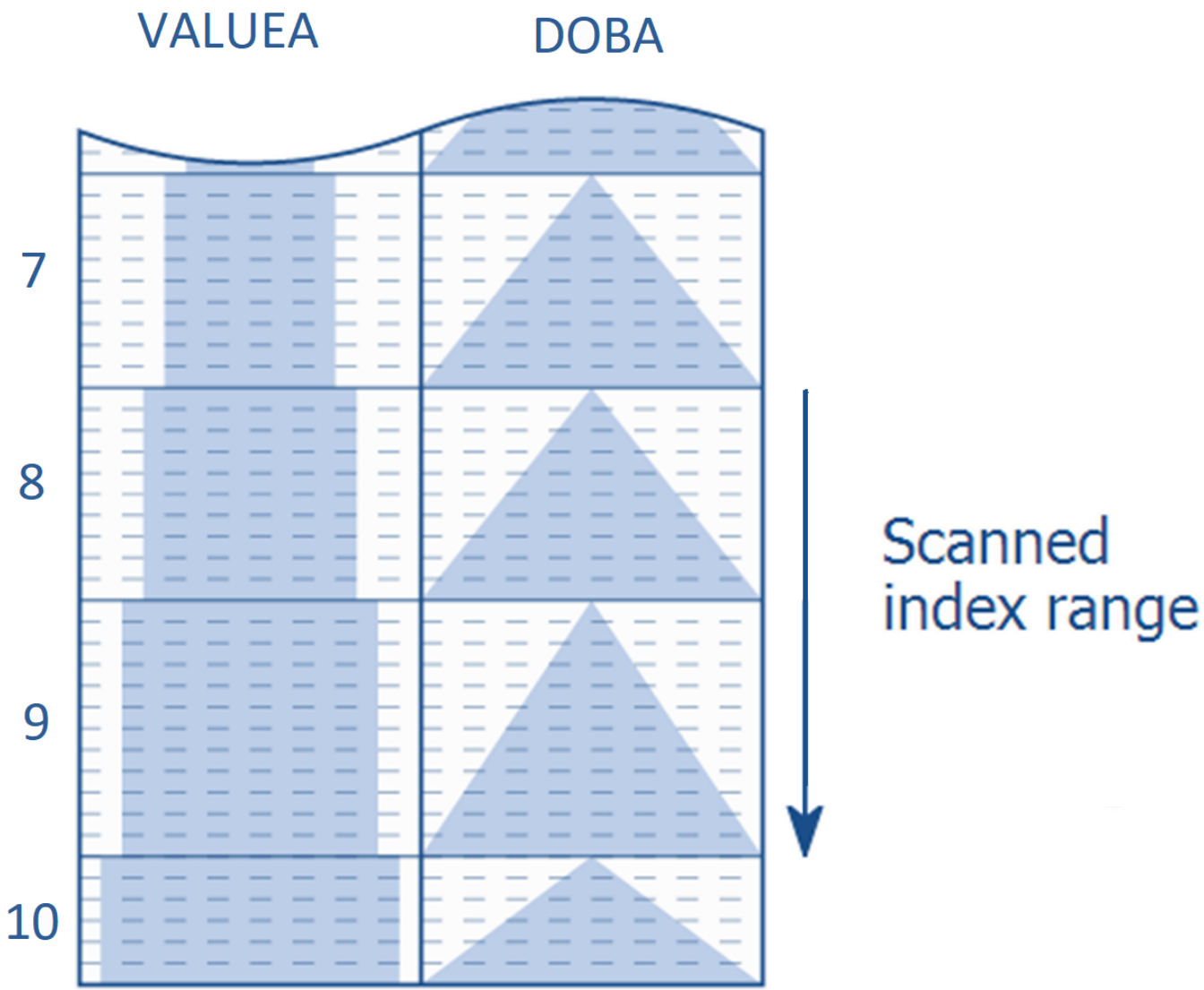
```
-----------------------------------------------------------------------------
| Id  | Operation                    | Name          | Rows  | Bytes | Cost (
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |               |   784K|  8424K|  2413
|*  1 |  INDEX RANGE SCAN DESCENDING| IDX_VALA_DOBA |   784K|  8424K|  2413
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("VALUEA"=9)
```

Indexarea pentru ORDER BY

- Ce se întamplă dacă mărim intervalul căutat ?

```
select valuea, doba from fg where valuea
  IN (8,9) order by doba;
```

Indexarea pentru ORDER BY



```
-----------------------------------------------------------------------------
| Id  | Operation              | Name          | Rows  | Bytes |TempSpc| Cost (%CPU
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |               |  1522K|    15M|       | 10929   (1
|   1 |  SORT ORDER BY         |               |  1522K|    15M|    29M| 10929   (1
|*  2 |   INDEX FAST FULL SCAN| IDX_VALA_DOBA |  1522K|    15M|       |  4274   (1
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
   2 - filter("VALUEA"=8 OR "VALUEA"=9)
```

Utilizează indexul pentru a găsi exact liniile ce trebuie întoarse dar acestea vor fi reordonate.
(pentru că în index nu sunt aşezate după doba).

A	B
1	a
1	b
1	c
1	d
2	a
2	b
2	c
2	d
3	a
3	b
3	c
3	d
4	a
4	b
4	c
4	d

```
CREATE INDEX exemplu ON tabel(A,B);

SELECT * FROM tabel
        WHERE A >1 AND A<4
        ORDER BY A,B;
```

Merge pe index pana gaseste inceputul intervalului unde A>1 si A<4, ajunge la cuplul (2,a) dupa care continua pe lista dublu inlantuita pana ajunge la (3,d).

A	B
1	a
1	b
1	c
1	d
2	a
2	b
2	c
2	d
3	a
3	b
3	c
3	d
4	a
4	b
4	c
4	d

```
CREATE INDEX exemplu ON tabel(A,B);

SELECT * FROM tabel
        WHERE A >1 AND A<4
        ORDER BY A,B desc;
```

Merge pe index pana gaseste inceputul intervalului unde A>1 si A<4, ajunge la cuplul (2,a). In continuare, daca ar merge pe lista dublu inlantuita nu ar putea furniza coloana B in ordine descrescatoare.

A	B
1	d
1	c
1	b
1	a
2	d
2	c
2	b
2	a
3	d
3	c
3	b
3	a
4	d
4	c
4	b
4	a

```
CREATE INDEX exemplu ON tabel(A,B desc);
```

Sau ?

```
CREATE INDEX exemplu ON tabel(A desc,B);
```

```
SELECT * FROM tabel
        WHERE A >1 AND A<4
        ORDER BY A,B desc;
```

A	B
4	a
4	b
4	c
4	d
3	a
3	b
3	c
3	d
2	a
2	b
2	c
2	d
1	a
1	b
1	c
1	d

A	B
1	d
1	c
1	b
1	a
2	d
2	c
2	b
2	a
3	d
3	c
3	b
3	a
4	d
4	c
4	b
4	a

```
CREATE INDEX exemplu ON tabel(A,B desc);
```

Sau ?

```
CREATE INDEX exemplu ON tabel(A desc,B);
```

Cand criteriul de ordonare este invers fata de ordinea existenta in index, acesta (indexul) este parcurs in sens invers…

A	B
4	a
4	b
4	c
4	d
3	a
3	b
3	c
3	d
2	a
2	b
2	c
2	d
1	a
1	b
1	c
1	d

Indexarea pentru ORDER BY

- Dacă BD foloseşte o operaţie de sortare chiar când vă aşteptaţi la o execuţie *pipelined,* motivul este unul din următoarele două:
  - QO consideră că execuţia este optimă în acest fel.
  - Ordinea indecşilor nu corespunde clauzei "ORDER BY"
- Puteţi să vă daţi seama de motiv folosind un *full index definition* în clauza *order by.*

Indexarea pentru ORDER BY

- Atunci cand totusi QO prefera sa sorteze lista, acest lucru se intampla pentru ca prefera sa faca operatia peste toata lista (de exemplu pentru ca oricum trebuie sa incarce toate inregistrarile) si ordonarea nu i-ar lua mai mult timp decat daca ar citi din indecsi si ar accesa tabela pentru fiecare rowid.

ASC/DESC/NULL FIRST,LAST

Applies to
DB2 ✓
MySQL ✗
Oracle ✓
PostgreSQL ✓
SQL Server ✓

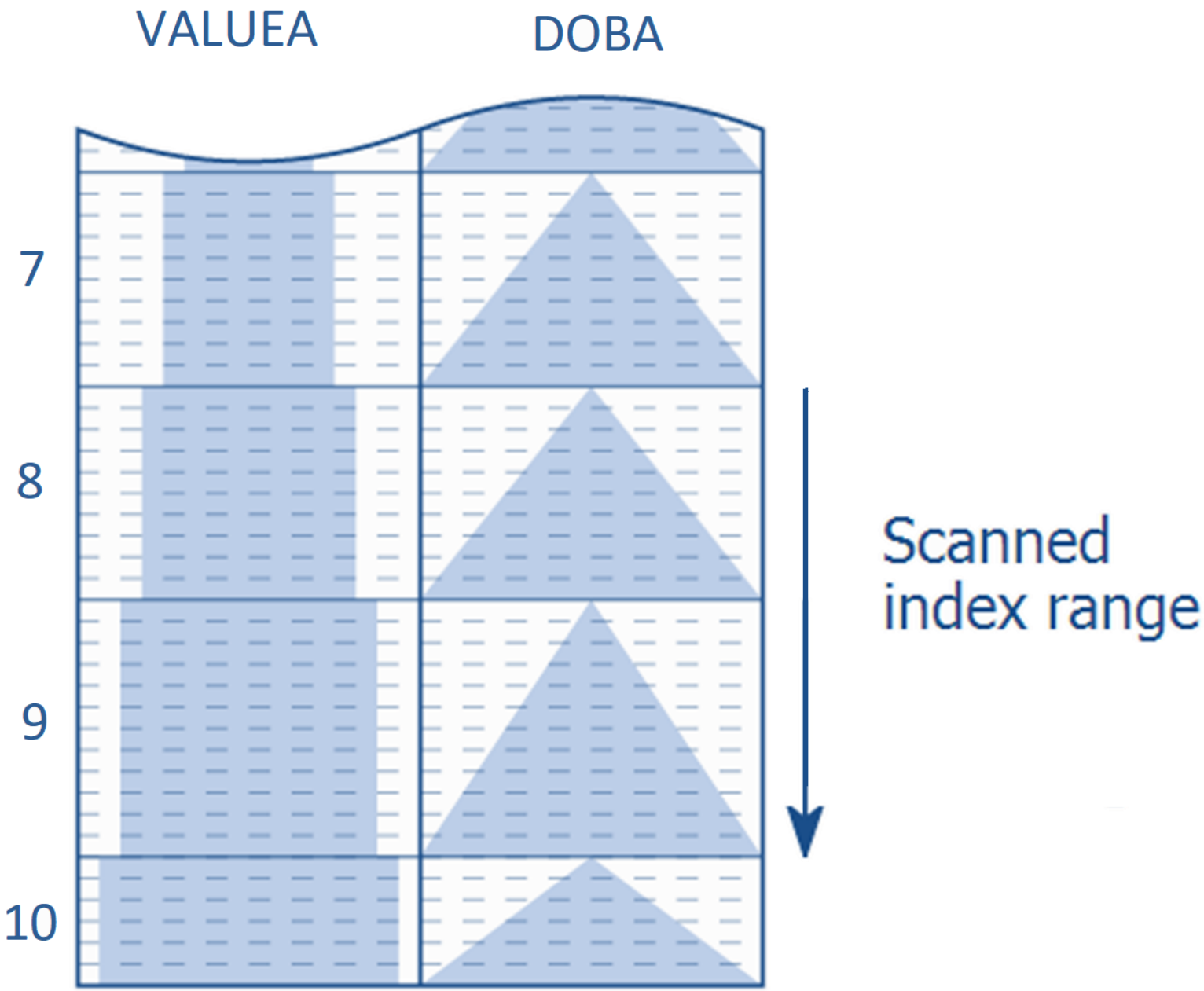
- BD pot citi indecşii în ambele direcţii.
- Indecşii sunt folosiţi şi dacă ORDER BY trebuie să întoarcă rezultatul în ordine inversă.
- Modificatorii ASC/DESC **pot** schimba execuţia de tip pipedline într-una în care trebuie făcută sortarea.
- Avem indexul peste **`fg(valuea,doba)`**
- **`dobA`** poate fi NULL…

ASC/DESC/NULL FIRST,LAST

```
select valuea, doba from fg where valuea
  between 8 and 9 order by valuea,doba;
```

- În acest caz, planul de execuţie arată astfel:

```
-----------------------------------------------------------------------
| Id  | Operation         | Name          | Rows  | Bytes | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |               |  1522K|    15M|  4508   (1)|
|   1 |  INLIST ITERATOR  |               |       |       |            |
|*  2 |   INDEX RANGE SCAN| IDX_VALA_DOBA |  1522K|    15M|  4508   (1)|
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("VALUEA"=8 OR "VALUEA"=9)
```

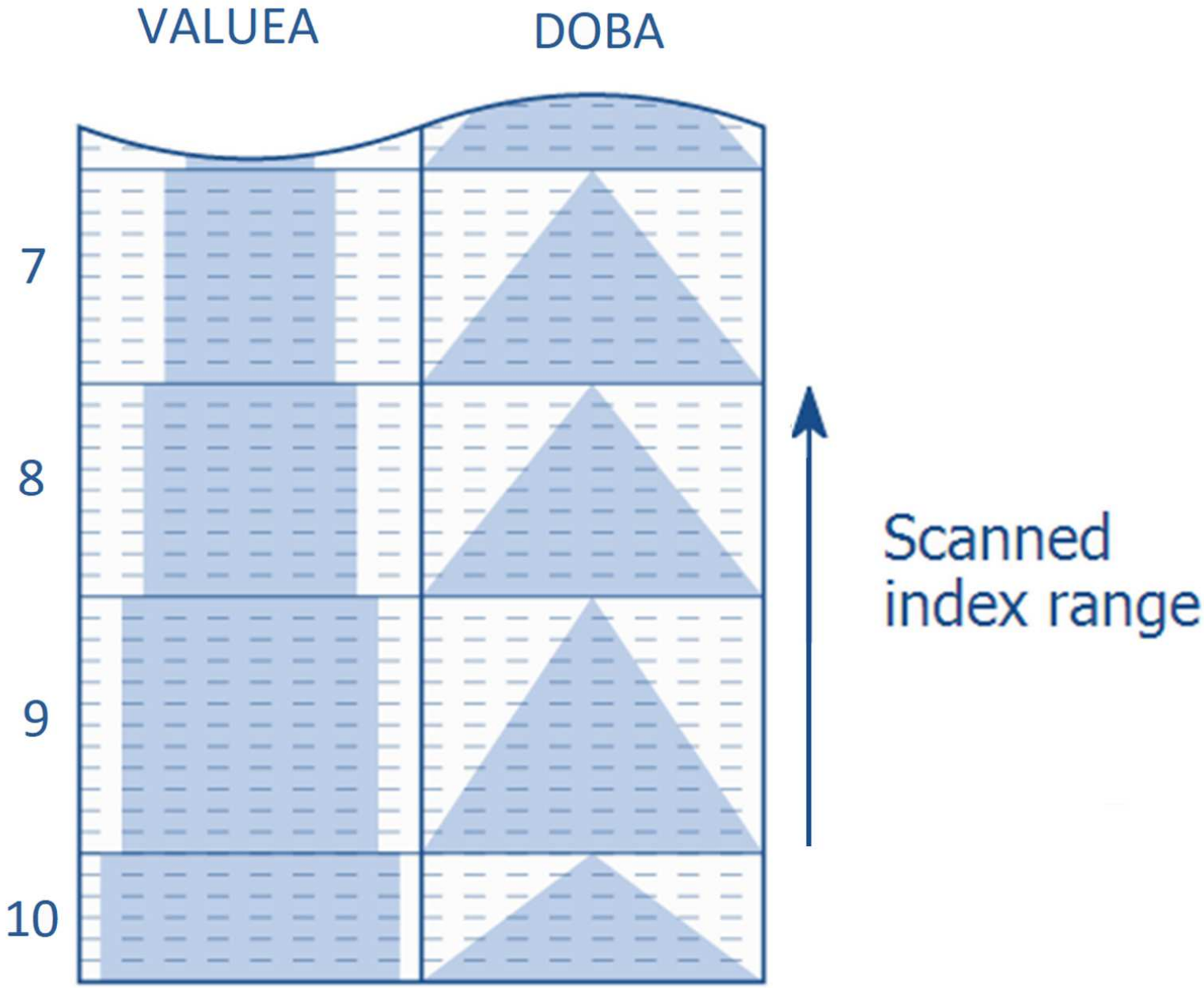

VALUEA
DOBA
7
8
9
10
Scanned
index range

ASC/DESC/NULL FIRST,LAST

```
select valuea, doba from fg where valuea
  between 8 and 9 order by valuea desc,doba
  desc;
```

- În acest caz, planul de execuţie arată astfel:

```
-----------------------------------------------------------------------
| Id  | Operation                    | Name          | Rows  | Bytes | Cost (%
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT             |               |  1522K|    15M|  4508
|   1 |  INLIST ITERATOR             |               |       |       |
|*  2 |   INDEX RANGE SCAN DESCENDING| IDX_VALA_DOBA |  1522K|    15M|  4508
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("VALUEA"=8 OR "VALUEA"=9)
```

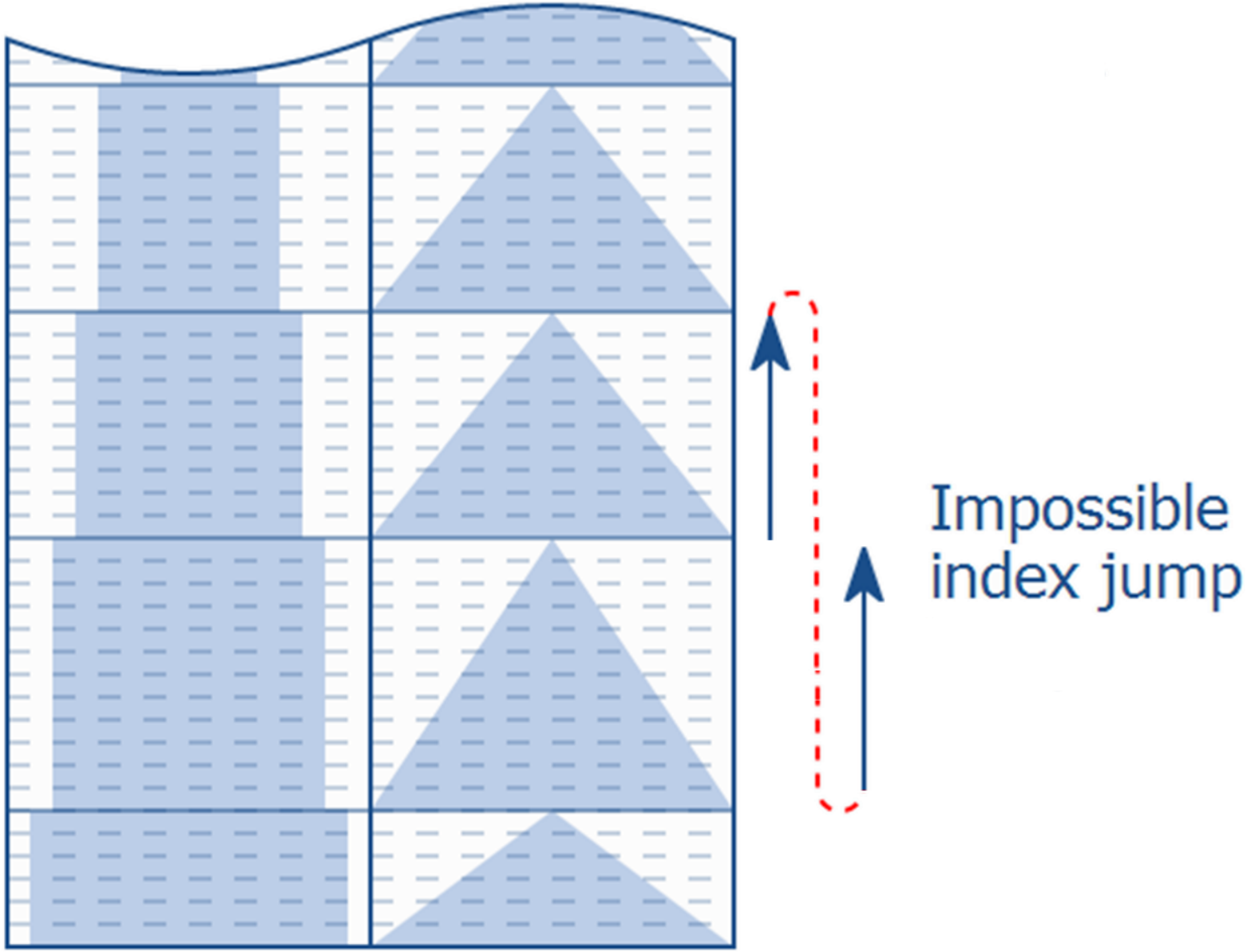

VALUEA
DOBA
7
8
9
10
Scanned
index range

ASC/DESC/NULL FIRST,LAST

```
select valuea, doba from fg where valuea
  between 8 and 9 order by valuea asc,doba
  desc;
```

- În acest caz, planul de execuţie arată astfel:

```
-----------------------------------------------------------------------------
| Id  | Operation              | Name          | Rows  | Bytes |TempSpc| Cost (%C
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |               |  1522K|    15M|       | 10929
|   1 |  SORT ORDER BY         |               |  1522K|    15M|    29M| 10929
|*  2 |   INDEX FAST FULL SCAN| IDX_VALA_DOBA |  1522K|    15M|       |  4274
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
   2 - filter("VALUEA"=8 OR "VALUEA"=9)
```


Impossible
index jump

ASC/DESC/NULL FIRST,LAST

- Este crucial ca indexul să fie EXACT

  - în aceeaşi ordine cu ce se cere în ORDER BY

  - în ordine inversă faţă de ce se cere în clauza ORDER BY!

ASC/DESC/NULL FIRST,LAST

- Pentru cazuri ca cel precedent, cele mai multe BD oferă posibilitatea de a crea indecşi într-o anumită ordine.

ASC/DESC/NULL FIRST,LAST

```
create index idx_vala_doba on
        fg(valuea desc ,doba asc);
```

Eu am ales să
îl fac pe acesta.

SAU

```
create index idx_vala_doba on
        fg(valuea asc, doba desc);
```

- [MySQL ignora ASC/DESC din definiţia indecşilor] ☹
- Noua definiţie nu afectează căutarea în clauza WHERE.

```
select valuea, doba from fg
where valuea between 8 and 9
order by valuea desc, doba asc;
```

ASC/DESC/NULL FIRST,LAST

- Dupa crearea noului index, se va evita sortarea (din nou):

```
-----------------------------------------------------------------------
| Id  | Operation        | Name          | Rows  | Bytes | Cost (%CPU)|
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT |               |  1522K|    15M|    76   (0)|
|*  1 |  INDEX RANGE SCAN| IDX_VALA_DOBA |  1522K|    15M|    76   (0)|
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access(SYS_OP_DESCEND("VALUEA")>=HEXTORAW('3EF5FF')  AND
              SYS_OP_DESCEND("VALUEA")<=HEXTORAW('3EF6FF') )
       filter(SYS_OP_UNDESCEND(SYS_OP_DESCEND("VALUEA"))>=8 AND
              SYS_OP_UNDESCEND(SYS_OP_DESCEND("VALUEA"))<=9)
```

ASC/DESC/NULL FIRST,LAST

- ASC/DESC sunt utilizate pentru a inversa ordinea anumitor coloane. Nu este nevoie să le sortam pe toate DESC. De ce ?

ASC/DESC/NULL FIRST,LAST

- Inafară de ASC/DESC, SQL permite înca doi modificatori pentru ORDER BY (NULL FIRST sau NULL LAST).
- Problema apare din cauza ca nu se poate specifica ordinea pentru NULL (în index).
- Nu este implementată nici de SQL server 2012 şi nici de MySQL 5.6.
- Oracle SQL suporta sortarea NULL-urilor înainte sa fie introdus în standard dar nu permite indexarea lor nici macar în *11g*.

ASC/DESC/NULL FIRST,LAST

- Oracle nu va permite *pipeline* atunci cand se va face sortare cu NULLS FIRST.
- PostgreSQL suporta (din ver 8.3) modificatori NULLS atât în ORDER BY cât şi în definirea indecşilor.

ASC/DESC/NULL FIRST,LAST

	DB2	MySQL	Oracle	PostgreSQL	SQLite	SQL Server
Read index backwards	✓	✓	✓	✓	✓	✓
Order by ASC/DESC	✓	✓	✓	✓	✓	✓
Index ASC/DESC	✓	✗	✓	✓	✓	✓
Order by NULLS FIRST/LAST	✗	✗	✓	✓	✗	✗
Default NULLS order	First	First	Last	Last	First	First
Index NULLS FIRST/LAST	✗	✗	✗	✓	✗	✗

Idee de studiu individual: demonstraţi că Oracle indexează default NULL la sfârşit.

Indecşii în grupare (GROUP BY)

Indexarea GROUP BY

Applies to
DB2 ✓
MySQL ✓
Oracle ✓
PostgreSQL ✓
SQL Server ✓

- BD SQL folosesc doi algoritmi diferiti:

1) algoritm **Hash** care face agregare de înregistrări într-o tabela hash. După ce toate inputurile au fost procesate, se returnează tabela hash (nu şi in MySQL 5.6)

2) **sort/group**: întâi sortează datele după cheile de grupare a.i. ele sa fie consecutive după care le face agregare.

Indexarea GROUP BY

- Ambii algoritmi au paşi intermediari (construire de **hashuri** / **sortarea**) şi din acest motiv nu pot fi executaţi în maniera *pipelined*.
- Totuşi, algoritmul sort/group (al doilea) poate utiliza un index pentru a evita sortarea, în acest fel dând posibilitatea de efectuare a *pipeline* chiar şi în cadrul grupărilor.

Indexarea GROUP BY

- Pentru a se produce un *pipedlined group by*, trebuie să fie indeplinite aceleaşi condiţii ca şi în cazul lui *order by*.
- Aceeaşi observaţie pentru NULLS FIRST/LAST.
- Există baze de date care nu pot procesa corect ASC/DESC indexarea pentru a face *pipelined group by* [In PostgreSQL trebuie adaugat un order by pentru a face un NULLS LAST ce va fi folosit in group by].

Indexarea GROUP BY

- Pentru a returna liniile în ordine inversă, indexul putea fi parcurs invers (în cazul ORDER BY).
- În Oracle **nu** se poate citi un index în ordine inversă pentru a se executa un *piped group by* dacă este urmat de un *order by*.

Primele *n* înregistrări

Rezultate parţiale

- Uneori nu sunt necesare toate liniile unei interogari SQL ci doar primele *n* (câteva) linii [fie pentru o afişare paginată fie pentru un "scroll infinit"].
- Asta ar putea crea probleme de performanţă dacă toate inregistrările trebuie sortate pentru a fi returnate doar primele *n*. Din acest motiv, un "***pipelined ordered by***" este o optimizare puternică.

Rezultate parţiale

- Un *pipelined* ORDER BY nu este atat de important din cauza ca poate sa returneze randurile sortate, ci mai degraba pentru ca poate sa returneze primele *n* randuri fara sa treaca prin toate (***pipelined* ORDER BY** are cost mic de pornire).

Primele *n* inregistrari

- Atunci cand se doreste preluarea primelor *n* inregistrari, QO nu isi da seama ca se doresc numai acele *n*.
- QO ar trebui sa stie daca in final aplicatia va prelua toate inregistrarile – atunci un *full scan* urmat de sortare ar fi cea mai buna solutie. Chiar si asa, un *piped order by* ar aduce un +.
- Tip: *spune-i BD cand nu ai nevoie de toate inregistrarile* - fetch first.

Primele *n* inregistrari

- **fetch first** – in SQL:2008, disponibil in IBM DB2, PostgreSQL, SQL Server 2012, Oracle 12c.

```
SELECT val FROM rownum_order_test
   ORDER BY val DESC
   FETCH FIRST 5 ROWS ONLY;
```

[Oracle 12c]

Primele *n* inregistrari

- In Oracle *11g*…

- Sa consideram urmatoarea interogare Oracle:

```
SELECT * FROM (
    SELECT *
    FROM fg WHERE valuea=9
    ORDER BY doba DESC )
WHERE rownum <= 10;
```

Primele *n* inregistrari

Si-a dat seama ca trebuie sa numere doar 10.

```
-----------------------------------------------------------------------------
| Id  | Operation                    | me  | Rows  | Bytes |TempSpc| Cost (%
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |     |    10 |    90 |       | 16781
|*  1 |  COUNT STOPKEY               |     |       |       |       |
|   2 |   VIEW                       |     |  784K| 6892K|       | 16781
|*  3 |    SORT ORDER BY STOPKEY|     |  784K| 8424K|    15M| 16781
|*  4 |     TABLE ACCESS FULL        | FG  |  784K| 8424K|       | 13352
-----------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(ROWNUM<=10)
   3 - filter(ROWNUM<=10)
   4 - filter("VALUEA"=9)
```

Primele *n* inregistrari

- BD poate optimiza interogarea doar daca stie de la inceput ce rezultat partial este asteptat.
- Va prefera sa utilizeze un *pipelined order by*:
- Utilizarea unei sintaxe corecte este numai jumatate din *job*.
- Pentru ca sa poata numara doar primele 10 inregistrari, trebuie ca neaparat sa aiba un *order by* care sa poata fi *piped* (altfel ar trebui sa sorteze tot).

Primele *n* inregistrari

create index idx_doba on fg(valuea,doba);

```
-----------------------------------------------------------------------
| Id  | Operation          | Name     | Rows  | Bytes | Cost (%CPU)| Tim
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT   |          |    10 |    90 |     3   (0)| 00:
|*  1 |  COUNT STOPKEY     |          |       |       |            |
|   2 |   VIEW             |          |    10 |    90 |     3   (0)| 00:
|*  3 |    INDEX RANGE SCAN| IDX_DOBA |    10 |   110 |     3   (0)| 00:
-----------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(ROWNUM<=10)
   3 - access("VALUEA"=9)
```

Daca ar fi nevoit sa ia toate inregistrarile (si nu doar primele 10):

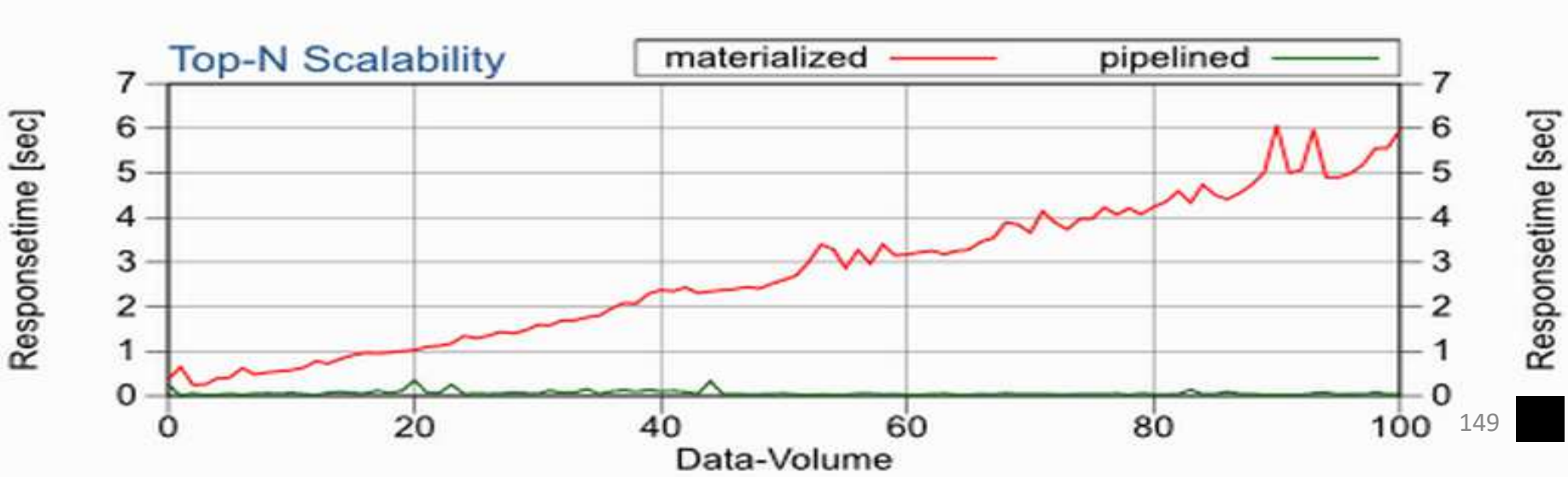
```
-----------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |   784K|  8424K|  2413   (1)| 00:00
|*  1 |  INDEX RANGE SCAN| IDX_DOBA |   784K|  8424K|  2413   (1)| 00:00
-----------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("VALUEA"=9)
```

Deci e clar că în cazul precedent, a “ştiut” să se oprească după primele 10
Asta din cauză că în timp ce face pipe, le şi numără.

Primele *n* inregistrari

http://blog.fatalmind.com/2010/09/29/finding-the-best-match-with-a-top-n-query/

- Avantajul nu este numai în performanţa imediată ci şi în scalabilitatea mărită. Fără *pipelined* execution, timpul creşte direct proporţional cu dimensiunea tabelei (când sunt *piped*, timpul este **aproape** identic).



Paginarea rezultatelor

Paginarea rezultatelor

- Ce se întâmplă după ce s-au întors primele *n* rezultate ?
- Avem nevoie de rezultatele [n+1 .. 2n] (pentru următoarea pagină).
- Există două metode:
  - *offset method* (numara liniile de la inceput, filtreaza dupa row number si face discard la ce e in plus)
  - *seek method* (cauta ultima intrare din pagina precedenta si preia randurile urmatoare)

Paginarea rezultatelor - *offset*

- Metoda offset utilizeaza un cuvant special "`offset`" – SQL Server standard ca si extensie pentru `fetch first`.

```
SELECT * FROM sales
      ORDER BY sale_date DESC
      OFFSET 10 ROWS
      FETCH NEXT 10 ROWS ONLY;
```

Paginarea rezultatelor - *offset*

- În Oracle e puţin mai complicat…

```
SELECT * FROM
      (SELECT tmp.*, rownum rn FROM
            (SELECT *
             FROM fg WHERE valuea=9
             ORDER BY doba DESC )
       tmp WHERE rownum <= 20)
WHERE rn > 10;
```

```
-----------------------------------------------------------------------------
| Id  | Operation                          | Name      | Rows  | Bytes | Cost (
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |           |    20 |  2440 |     4
|*  1 |  VIEW                              |           |    20 |  2440 |     4
|*  2 |   COUNT STOPKEY                    |           |       |       |
|   3 |    VIEW                            |           |    20 |  2180 |     4
|   4 |     TABLE ACCESS BY INDEX ROWID    | FG        |  784K|   47M|     4
|*  5 |      INDEX RANGE SCAN DESCENDING| IDX_DOBA |    20 |       |     3

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("RN">10)
   2 - filter(ROWNUM<=20)
   5 - access("VALUEA"=9)
```

Va trebui să ia 20 de rânduri şi să arunce primele 10.

Pe pagina *n* va lua *n**10 rânduri si va arunca (*n-1*)*10.

☹

Paginarea rezultatelor - *offset*

- Două dezavantaje:
  - pagina se *shiftează* când sunt adăugate înregistrari noi (pentru că o numără şi pe cea nou adăugată);
  - timpul de răspuns este mare când se accesează paginile cu număr mare.

Paginarea rezultatelor - *seek*

- Metoda *seek* utilizeaza valoarile din pagina anterioară ca punct de plecare şi evită ambele dezavantaje ale metodei *offset*.
- Caută înregistrările care urmează ultimei înregistrări de pe pagina anterioară (utilizând un simplu WHERE).

Paginarea rezultatelor - *seek*

- Presupunând că se pune o singură notă pe zi (deci data ar fi cheie unică):

```
SELECT * FROM fg
    WHERE doba < ?
    ORDER BY doba
    DESC FETCH FIRST 10 ROWS ONLY;
```

? = Ultimul sale_date de pe pagina precedenta

Paginarea rezultatelor - seek

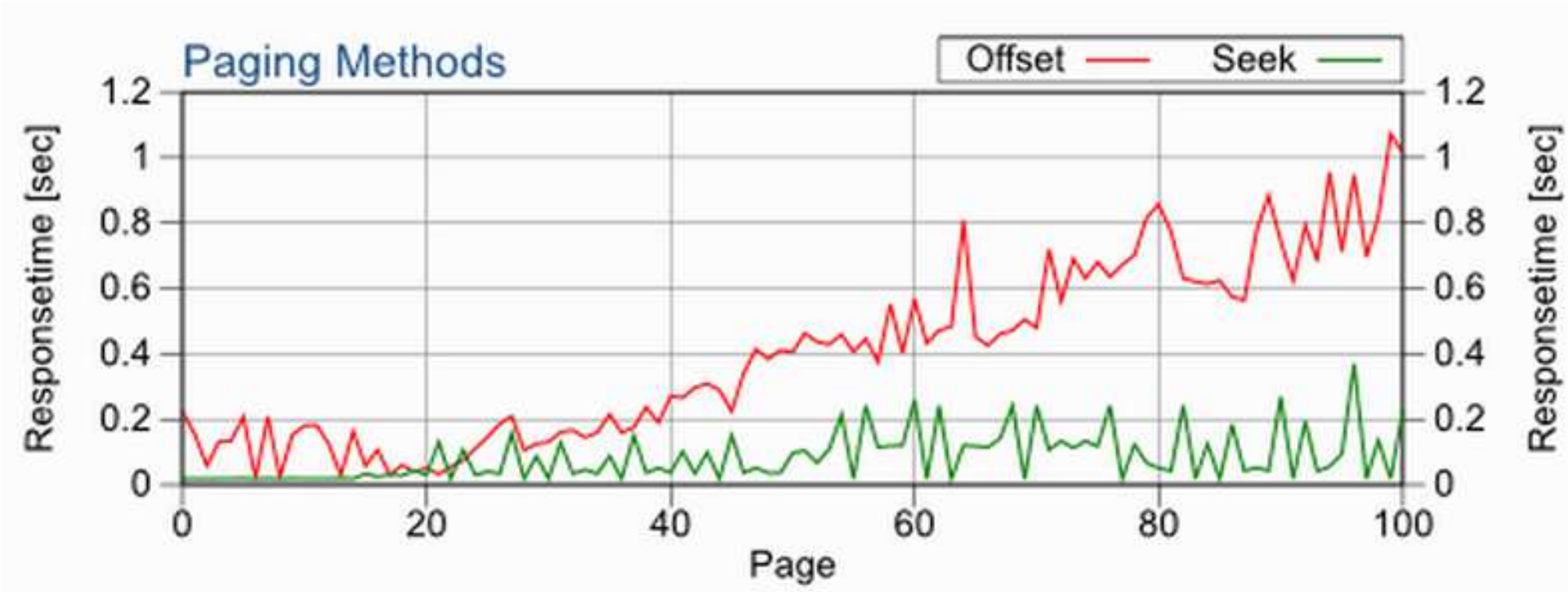
- Problema este ca dacă nu avem o **cheie unică** la sfârşit de pagină, se poate întâmpla să nu ştim de unde începem pagina următoare.
- Trebuie ca ***order by*** să fie **determinist*** altfel se poate întâmpla ca o nouă execuţie să facă un shuffle a înregistrărilor (din cauză că QO se hotărăşte să execute altfel interogarea şi atunci putem să ne trezim că s-a facut "*swap*" între ultima înregistrare de pe pagina anterioară şi prima de pe noua pagină).

*De exemplu nu ordonati dupa rownum care este o psudocoloana ce este construita de fiecare data la un nou apel.

Paginarea rezultatelor - seek

- Pentru a ne asigura că ORDER BY este mereu determinist, am putea să-l extindem (adăugând coloane) până ajungem la unicate (sau dacă nu se poate, se adaugă un câmp unic) – vezi “cheie candidat” de la BD.

Scalabilitate offset vs seek



Functii *Window* -> paginare

Applies to
DB2 ✗
MySQL ✗
Oracle ✓
PostgreSQL ✗
SQL Server ✓

- Metoda flexibila, conform standardului.
- Numai SQL server şi Oracle le utilizează pentru un *pipelined n-querry*
- PostgreSQL nu se opreşte din scanat după ce a preluat primele n linii ☹
- MySQL nu are deloc implementată o astfel de funcţie ☹

Functii *Window* -> paginare

rownum cu valorile dupa criteriul de ordonare

```
SELECT * FROM (
      SELECT fg.* , ROW_NUMBER() OVER
         (ORDER BY doba DESC ,
                              id DESC) rn
      FROM fg) tmp
 WHERE rn between 11 and 20
 ORDER BY doba DESC, id DESC;
```

Functii Window -> paginare

- ROW_NUMBER enumeră rândurile în funcţie de ordinea de sortare definită în clauza *over*.
- Ultimul *where* foloseşte valoarea lui *rn* pentru a prelua doar înregistrările 11-20.
- BD Oracle recunoaşte condiţia de oprire şi utilizează indecşii SALE_DATE si SALE_ID pentru a produce un comportament *pipelined*

Functii Window -> paginare

Deşi costul calculat este mare, timpul execuţiei este de 0 secunde !

```
---------------------------------------------------------------------------
 Id  | Operation                      | Name     | Rows  | Bytes | Cost (
---------------------------------------------------------------------------
   0 | SELECT STATEMENT               |          |  5102K|   593M| 67635
*  1 |  VIEW                          |          |  5102K|   593M| 67635
*  2 |   WINDOW NOSORT STOPKEY        |          |  5102K|   311M| 67635
   3 |    TABLE ACCESS BY INDEX ROWID| FG       |  5102K|   311M| 67635
   4 |     INDEX FULL SCAN DESCENDING| IDX_DOBA |  5102K|       | 17673
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("RN">=11 AND "RN"<=20)
   2 - filter(ROW_NUMBER() OVER ( ORDER BY INTERNAL_FUNCTION("DOBA") DESC
              ,INTERNAL_FUNCTION("ID") DESC )<=20)
```

Functii Window -> paginare

Operatie de tip NOSORT = pipelined

Ce se opreste cand se ajunge la o anumita valoare (stopkey)

```
-----------------------------------------------------------------------------
 Id  | Operation                                  | ows   | Bytes | Cost (
-----------------------------------------------------------------------------
   0 | SELECT STATEMENT                           |  5102K|   593M| 67635
*  1 |  VIEW                                      |  5102K|   593M| 67635
*  2 |   WINDOW NOSORT STOPKEY                    |  5102K|   311M| 67635
   3 |    TABLE ACCESS BY INDEX ROWID| FG         |  5102K|   311M| 67635
   4 |     INDEX FULL SCAN DESCENDING| IDX_DOBA   |  5102K|       | 17673
-----------------------------------------------------------------------------
'redicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("RN">=11 AND "RN"<=20)
   2 - filter(ROW_NUMBER() OVER ( ORDER BY INTERNAL_FUNCTION("DOBA") DESC
              ,INTERNAL_FUNCTION("ID") DESC )<=20)
```

Functii Window -> paginare

- Ce le face pe funcţiile window să fie importante nu este neapărat paginarea ci mai degrabă calculele analitice.

Insert, Update, Delete

Insert, Update, Delete

- Insert + Update + Delete = DML (Data Manipulation Language)
- Performanţa acestor comenzi este influenţată (negativ) de prezenţa indecşilor.
- Indexul = date redundante. Când sunt executate comenzile sus menţionate, indecşii trebuie refăcuti şi ei.

Insert

- Insertul este influenţat în primul rând de numărul indecşilor.
- Insertul nu are cum să beneficieze de pe urma indecşilor deoarece nu are o clauza “where”.

Insert

- Inserarea unui rand intr-o tabela presupune:

  - gasirea unei locatii pentru stocare [in tabelele fara indecsi, se cauta un block care are suficient spatiu si este adaugat acolo]

  - daca exista indecsi peste tabela, BD trebuie sa se asigure ca noua inregistrare apare in indecsi (va adauga cate o intrare in fiecare)

Insert

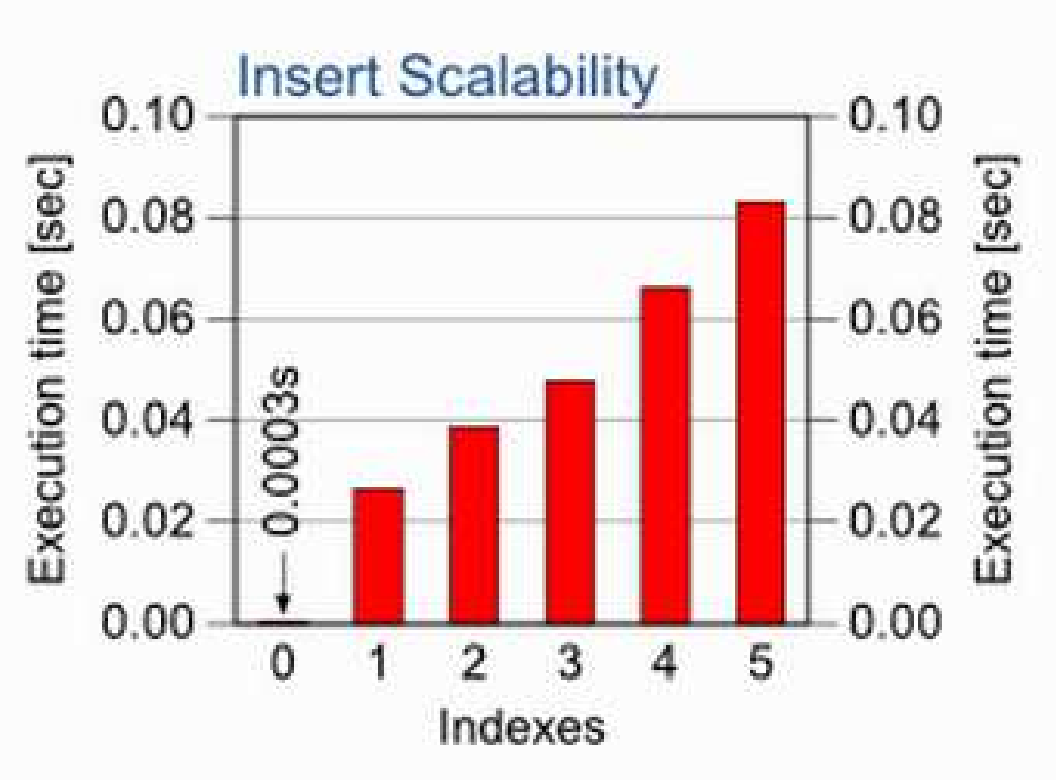
- adaugarea in index este mai costisitoare pentru ca arborele trebuie rebalansat. Din cauza existentei arborelui, inregistrarea nu poate fi scrisa in orice block; ea apartine unei anumite frunze din B-tree. Pentru a ajunge la locatie trebuie traversat arborele

Insert

- dupa indentificarea frunzei BD confirma daca mai exista spatiu in nodul respectiv. Daca nu mai exista, nodul este impartit in doua si distribuie intrarile intre vechiul si noul nod (aici apar referinte noi si in nodul superior care la randu-i poate fi impartit). Daca toate nodurile pana la radacina sunt impartite, arborelui i se mai adauga un nivel.

Insert

- Timpul unui insert in functie de numarul de indecsi:



Insert

- Primul index face cea mai mare diferenta
- Considerand ca intr-o tabela s-ar face numai inserturi, e mai bine fara indecsi.
- **Performanta fara indecsi este atat de buna ca uneori are sens sa renuntam la indecsi atunci cand incarcam cantitati mari de date (indecsii nu trebuie refacuti in timpul incarcarii ci numai o singura data la final).**

Insert

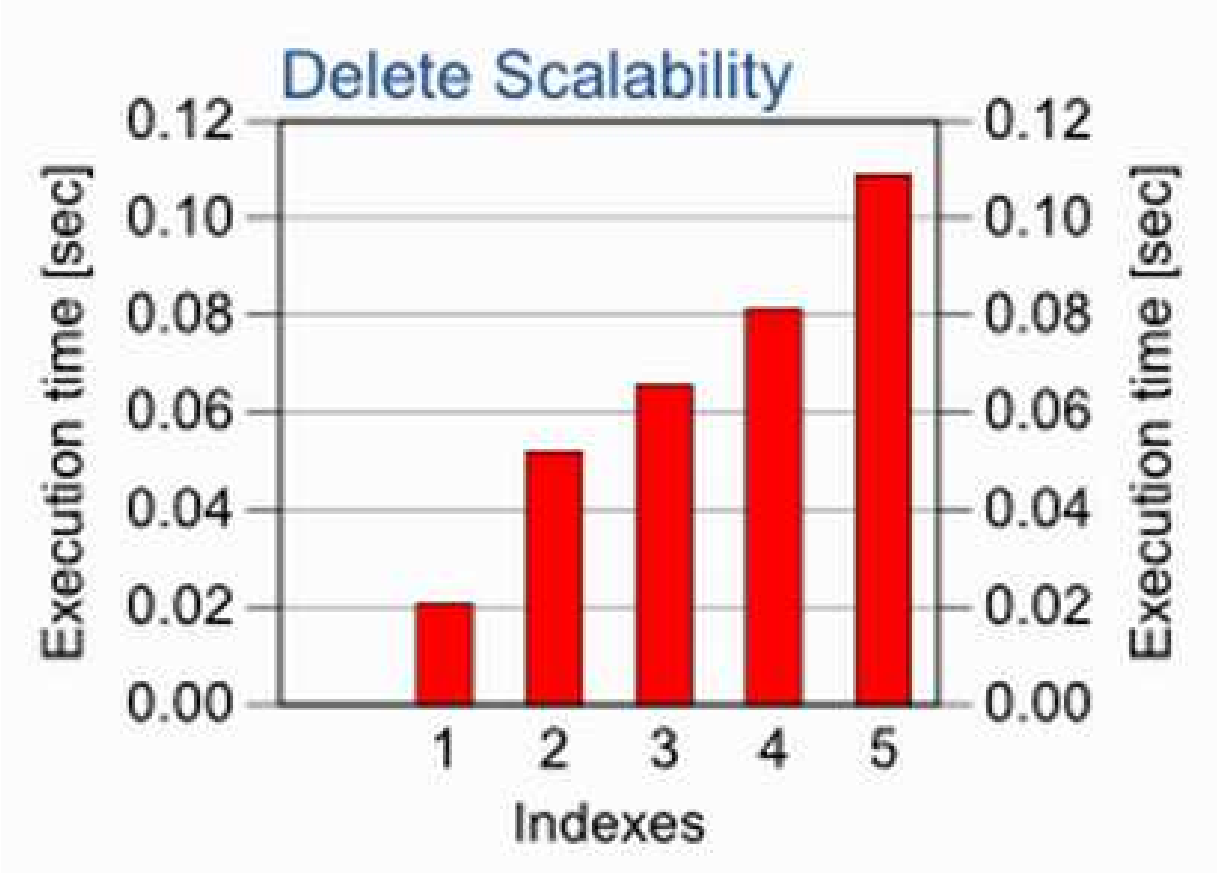
- Cum s-ar proceda in cadrul unei “index organized table” sau “clustered index” ?

Delete

- Delete are o clauza “where” care o face sa beneficieze de pe urma indecsilor. Funtioneaza ca un select urmat de stergerea datelor.
- Stergerea unui rand este inversa fata de inserarea unui rand.

Delete

- Eficienta operatiei delete este reprezentata in tabelul:



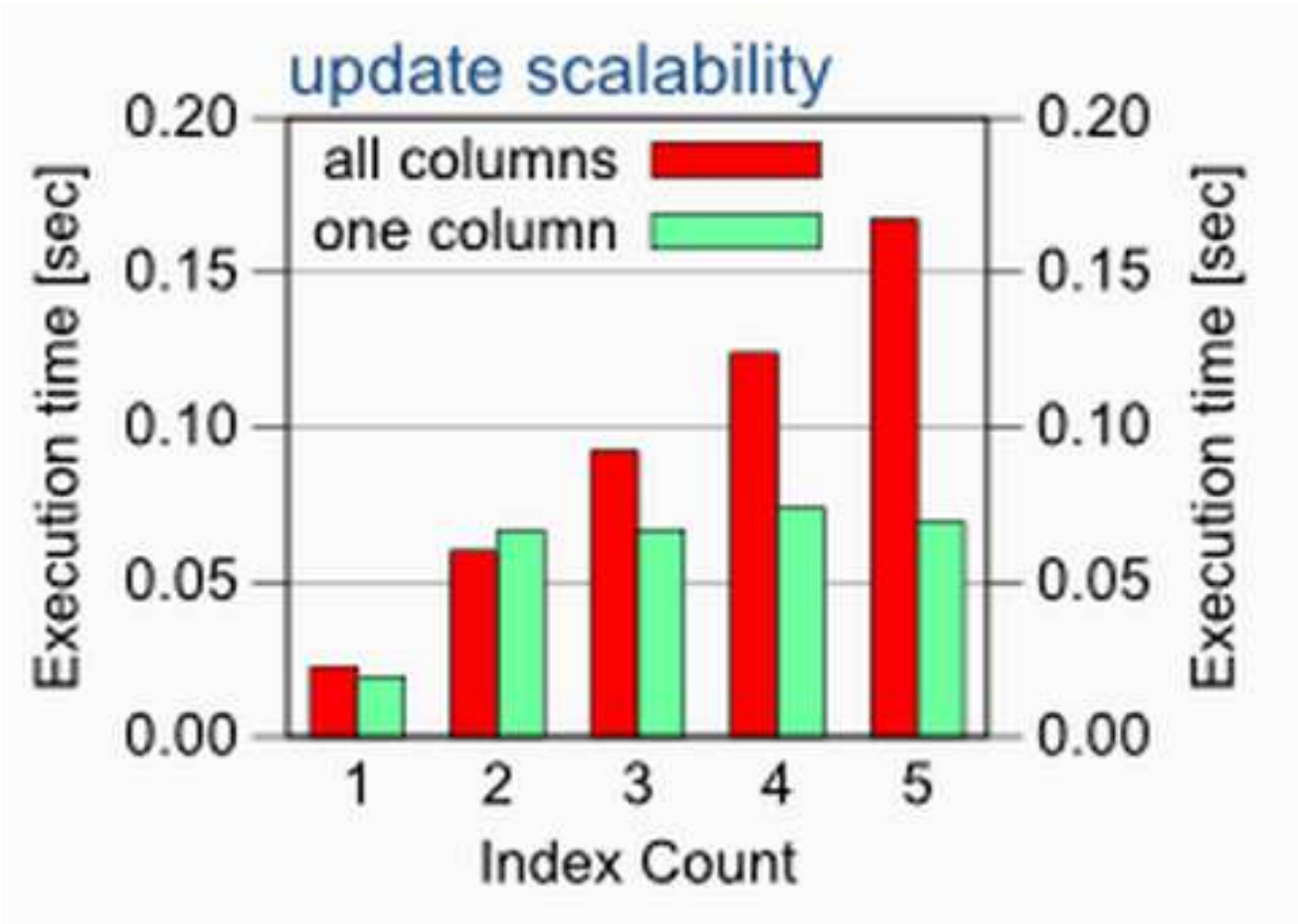
Delete

- Daca nu exista index, BD trebuie sa faca *full table scan* pentru a cauta inregistrarea.
- Are sens sa stergem fara “where” atunci cand sunt sterse foarte multe randuri.
- Operatiile delete si update au un execution plan (incercati acasa).

Update

- O operatie de tip Update trebuie sa realoce indexul (sa stearga vechea pozitie si sa realoce informatia in alta pozitie).
- Update (la fel ca insert si delete) depinde de numarul indecsilor peste tabela.
- Daca updateul nu modifica campurile din indecsi, timpul este minim (pentru ca indecsii nu trebuie refacuti).

Update


update scalability
all columns
one column
Execution time [sec]
0.20
0.15
0.10
0.05
0.00
1
2
3
4
5
Index Count

Update

- Daca este afectat un singur index (prin afectat se intelege ca trebuie sa fie re-aranjat), timpul ramane acelasi chiar daca sunt mai multi indecsi peste tabela.
- Cand se scrie comanda update, trebuie avut grija sa fie modificate doar acele coloane care trebuie modificate si nu toate coloanele (utilizand valori vechi in update).

Ganditi-va la un caz cand insert sau delete nu afecteaza indexul peste o tabela.
Raspuns:

Bibliografie (online)

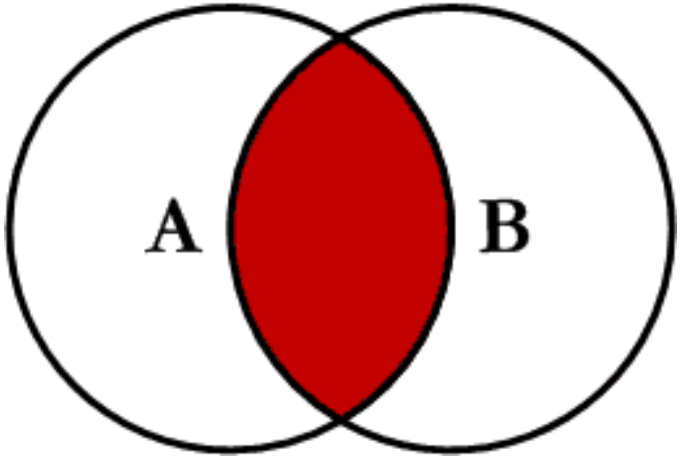
- http://use-the-index-luke.com/

  (puteti cumpara si cartea in format PDF – dar nu contine altceva decat ceea ce este pe site)

Joins – INNER JOIN

- http://www.codeproject.com/Articles/33052/Visual-Representation-of-SQL-Joins
- Cel mai simplu tip de join, interogarea va returna toate inregistrarile din A care au inregistrari pereche in tabela B.

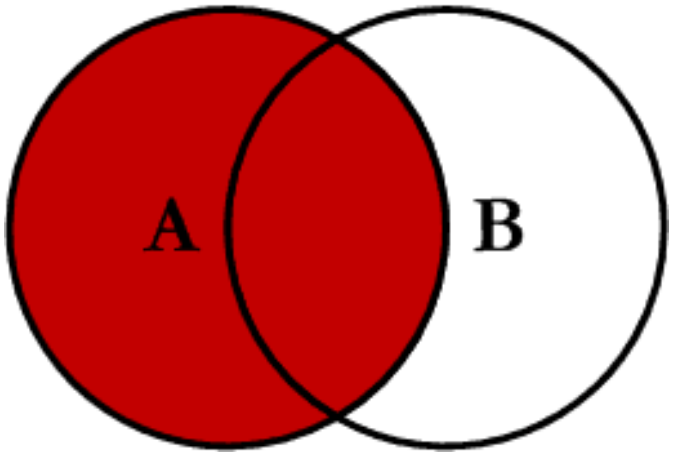
```
SELECT <select_list>
  FROM Table_A A
  INNER JOIN Table_B B
  ON A.Key = B.Key
```



Joins – LEFT JOIN

- Returneaza inregistrarile din tabela A indiferent daca au un match in tabela B. Daca in tabela B sunt inregistrari care fac match, le va potrivi.

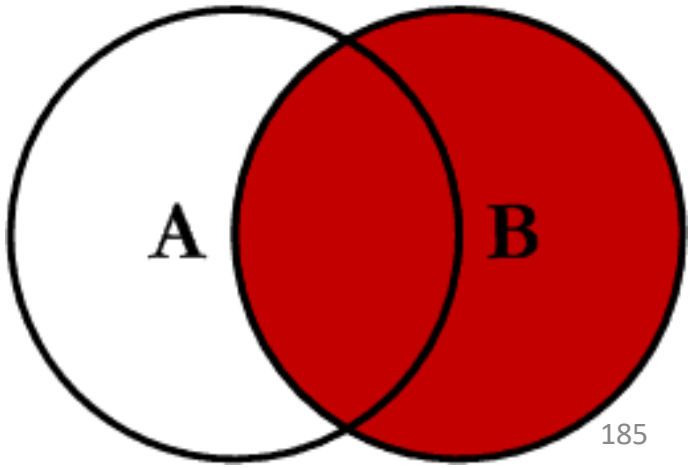
```
SELECT <select_list>
  FROM Table_A A
  LEFT JOIN Table_B B
  ON A.Key = B.Key
```



Joins – RIGHT JOIN

- Returneaza inregistrarile din tabela B indiferent daca au un match in tabela A. Daca in tabela A sunt inregistrari care fac match, le va potrivi.

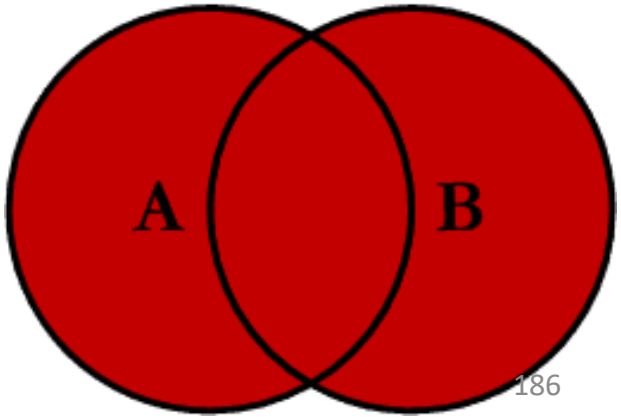
```
SELECT <select_list>
   FROM Table_A A
   RIGHT JOIN Table_B B
   ON A.Key = B.Key
```



Joins – OUTER JOIN

- Va intoarce inregistrarile care se potrivesc in ambele tabele (A si B).

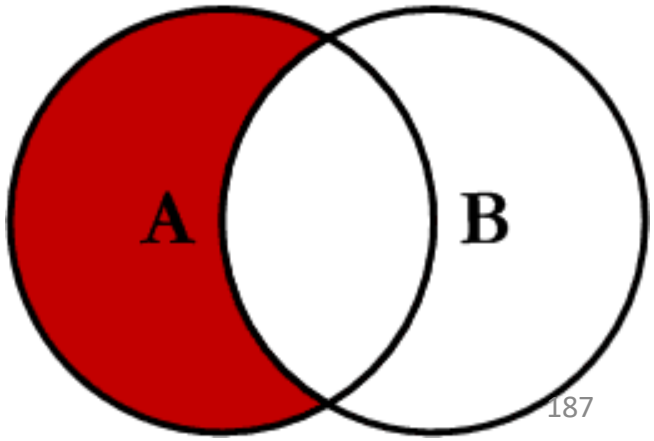
```
SELECT <select_list>
  FROM Table_A A
  FULL OUTER JOIN Table_B B
  ON A.Key = B.Key
```



Joins – LEFT EXCLUDING JOIN

- Va intoarce inregistrarile din A care nu au un match in B.

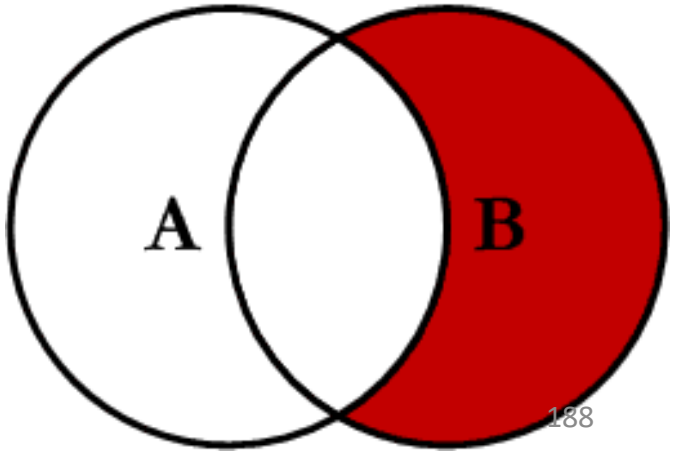
```
SELECT <select_list>
    FROM Table_A A
    LEFT JOIN Table_B B ON A.Key = B.Key
    WHERE B.Key IS NULL
```



Joins – RIGHT EXCLUDING JOIN

- Va intoarce inregistrarile din B care nu au un match in A.

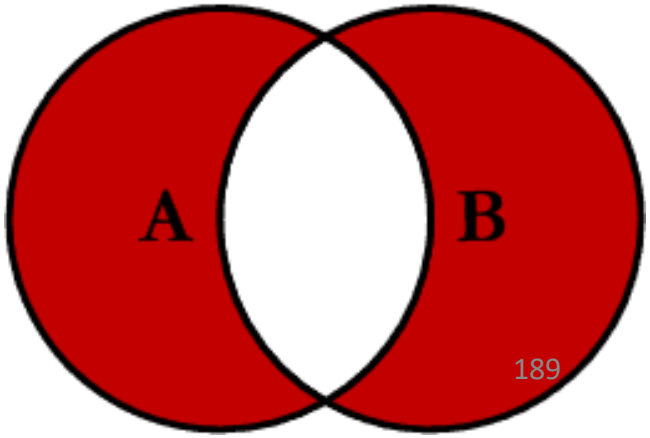
```
SELECT <select_list>
    FROM Table_A A
    RIGHT JOIN Table_B B
    ON A.Key = B.Key
    WHERE B.Key IS NULL
```



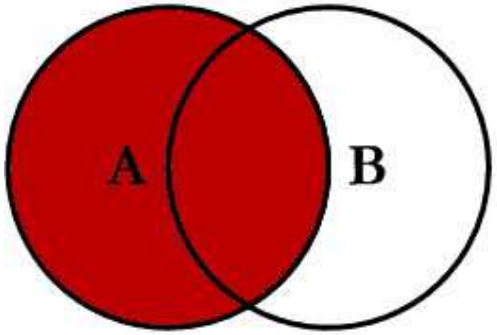
Joins – OUTER EXCLUDING JOIN

- Va intoarce inregistrarile din A si din B care nu se potrivesc.

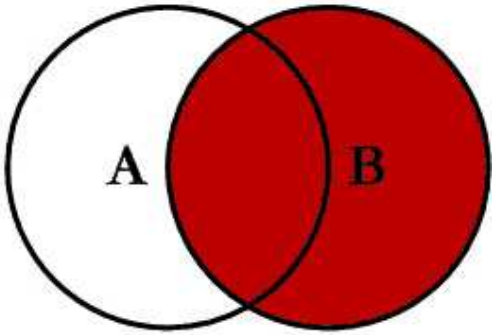
```
SELECT <select_list>
    FROM Table_A A FULL OUTER JOIN Table_B B
    ON A.Key = B.Key
    WHERE A.Key IS NULL OR B.Key IS NULL
```



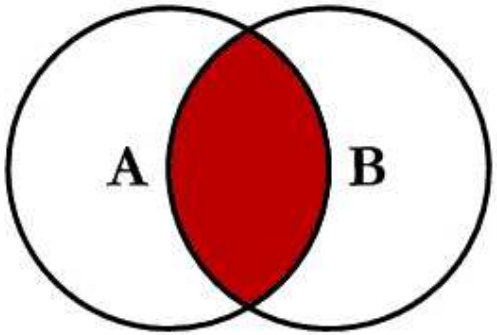
SQL JOINS



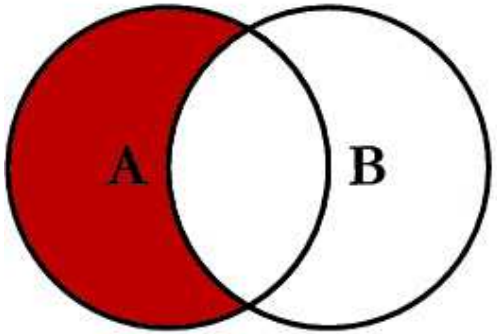

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



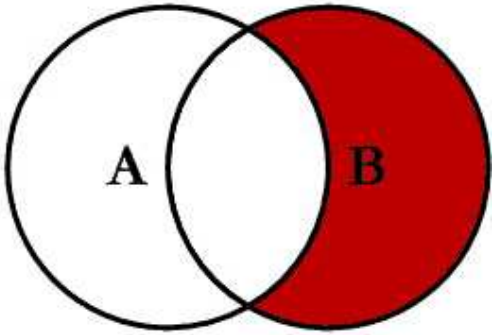

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



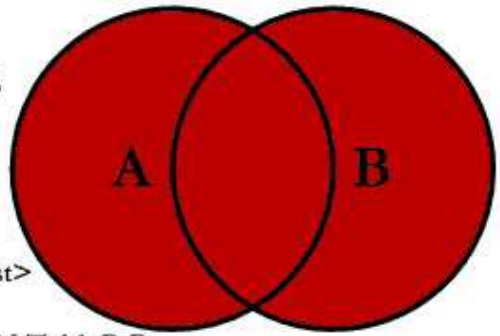

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



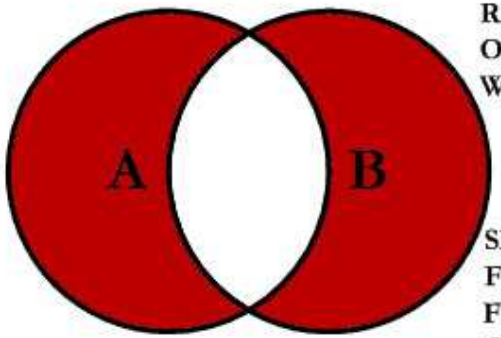

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008